UNIT 5

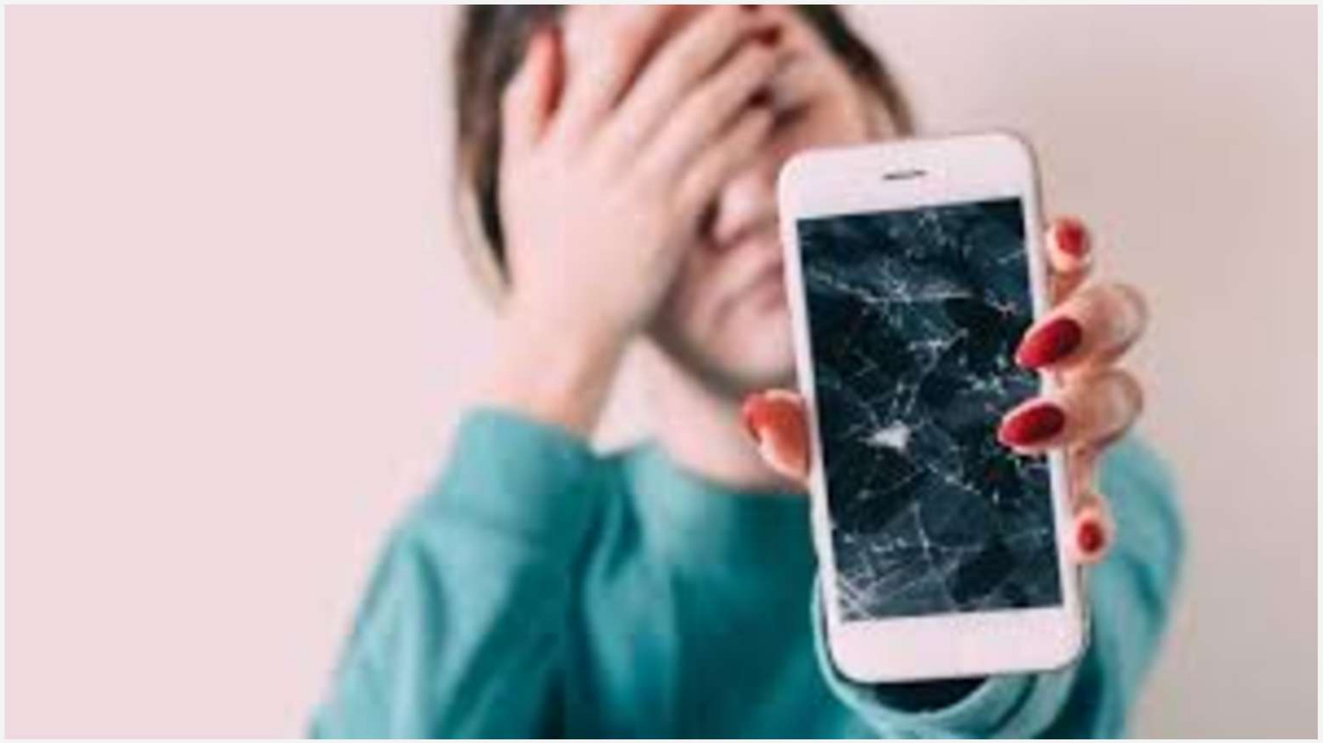
PERFORMANCE, SECURITY & SCALABILITY TESTING

Unit V
Performance, Security & Scalability Testing

Presented By:
Premitha Kamath

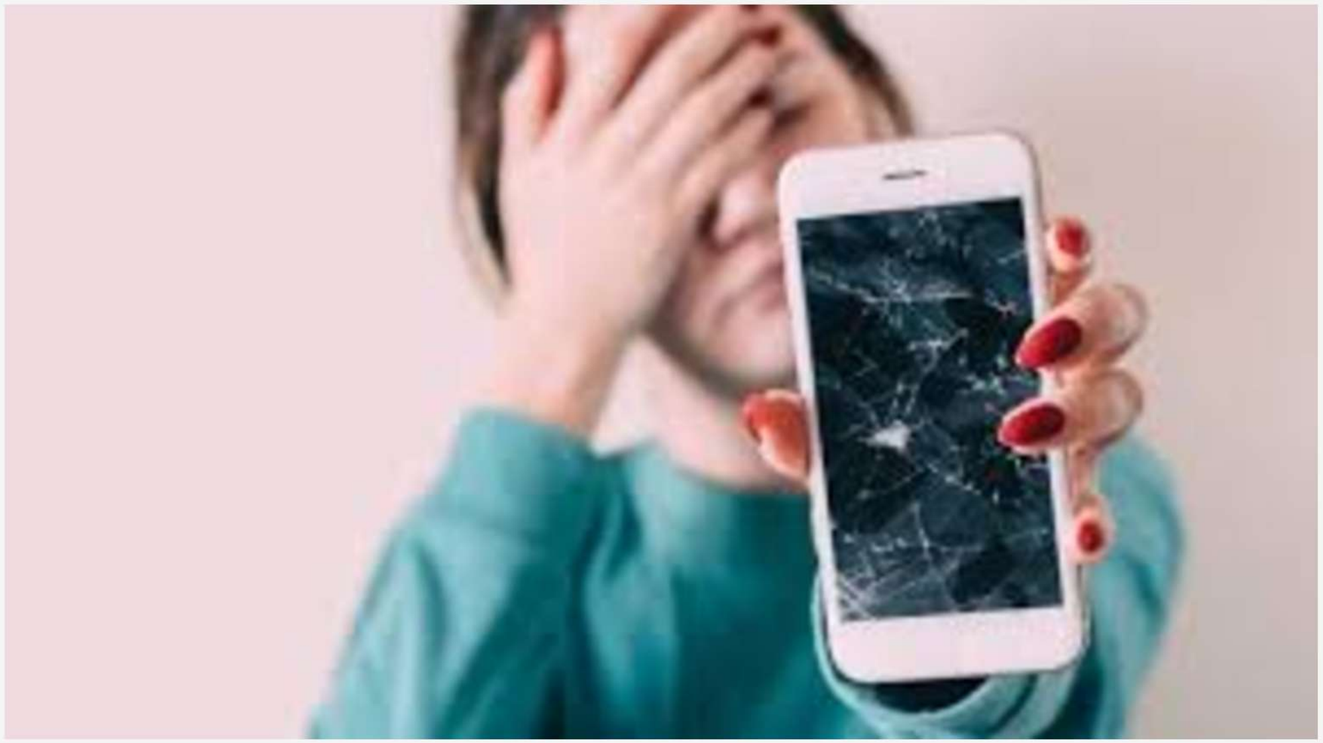
Presented To:
The class of MCA, NMAMIT

Lets buy a brand new phone!



Why did this happen?

How are you feeling about it ?



Why did this happen?

Would you trust the brand or the source again?

Product Testing

Recreate various test environments and check if the product can sustain it.

What's Software Testing then?

It is a pre-defined process that we need to follow step by step to ensure the software correctly meets the requirements without any errors.

What is software Testing?

Software testing is the process of checking and evaluating software to ensure it works correctly, meets requirements, and has no defects (bugs) before it is released or used.

Types

- **Manual Testing:** Testers run the software manually to find issues.
- **Automated Testing:** Tools and scripts automatically test the software.
- **Functional Testing:** Checks if features work as expected.
- **Non-Functional Testing:** Tests performance, security, usability, etc.
- **Regression Testing:** Ensures new changes don’t break existing features.

Performance vs. Functional Testing

In the software development lifecycle, "working" is not the same as "performing."

Functional Testing

Validates that the software performs its intended actions.

- **Question:** Can a user log in?
- **Result**: Pass or Fail.

Performance Testing

Evaluates the speed, responsiveness, and stability of the system under a particular workload.

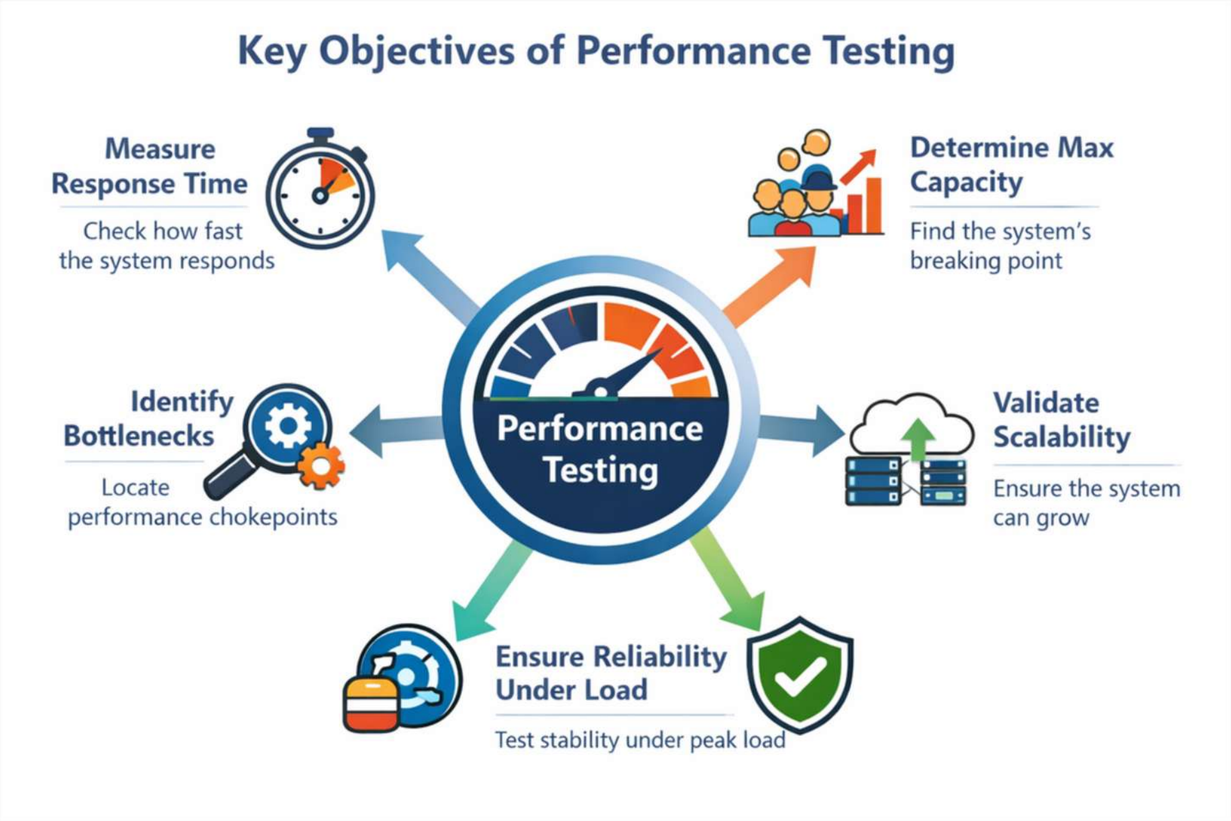
- **Question:** How long does the login take for 1,000 concurrent users?
- **Result:** A set of metrics (seconds, MB, %) compared against a baseline.

Performance Testing

Performance Testing is a type of software testing that ensures software applications perform properly under their expected workload. It is a testing technique carried out to **determine system performance in terms of sensitivity, reactivity, and stability under a particular workload.**

Objectives of Performance Testing

- Measure system response time
- Determine maximum system capacity
- Identify performance bottlenecks
- Ensure system reliability under load
- Validate system scalability



Key Performance Indicators (KPIs)

To measure performance, we use four primary metrics:

Response Time

Response time is the total time taken from when a user sends a request to when the system returns the final response.

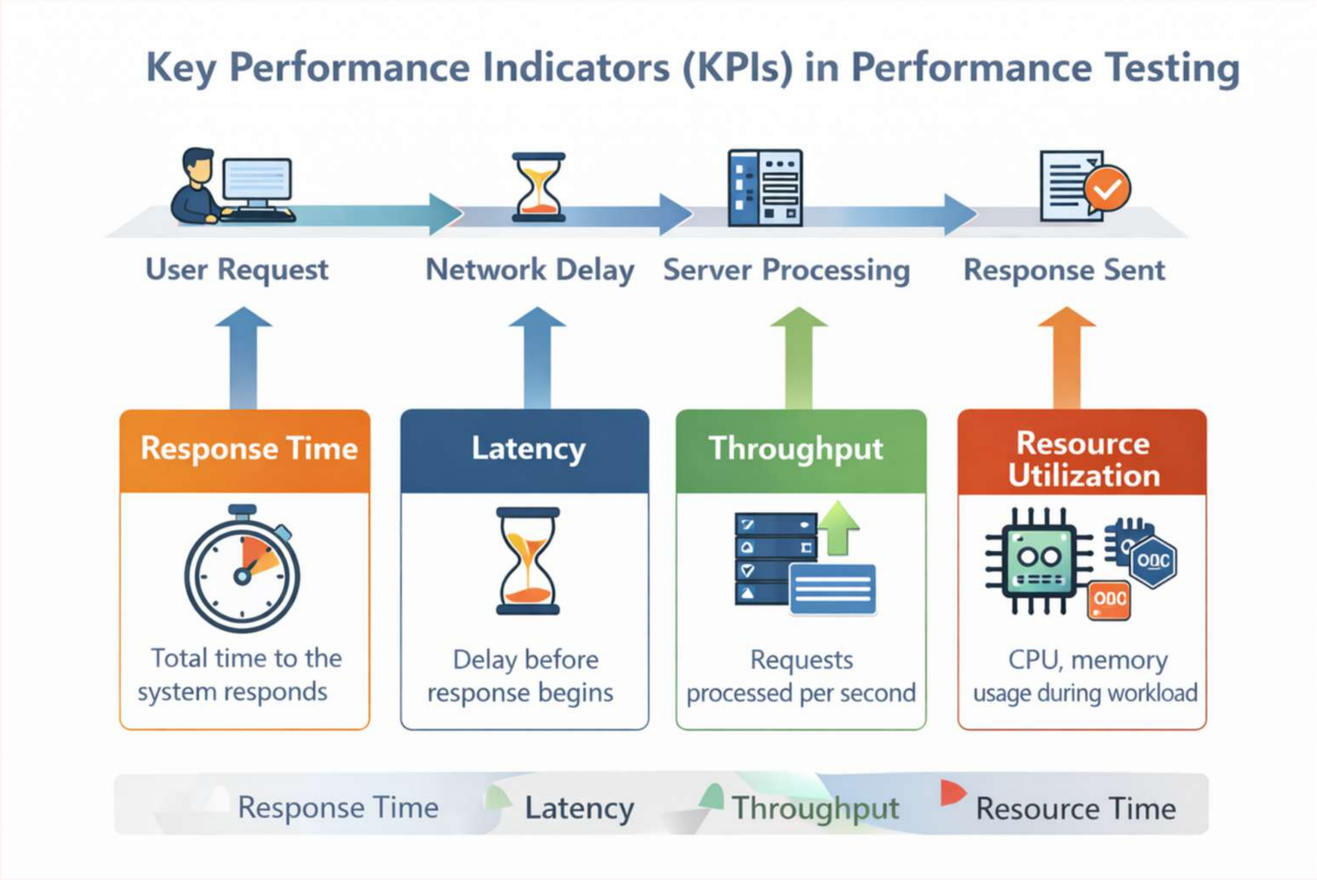
Latency

Latency refers to the delay before the system begins to respond to a request. In networking terms, it is the time taken for data packets to travel from the source to the destination. Latency is sometimes called network delay.

Throughput

Throughput measures the number of requests or transactions processed by the system per unit of time. It indicates the capacity of the system.
It is often measured in:

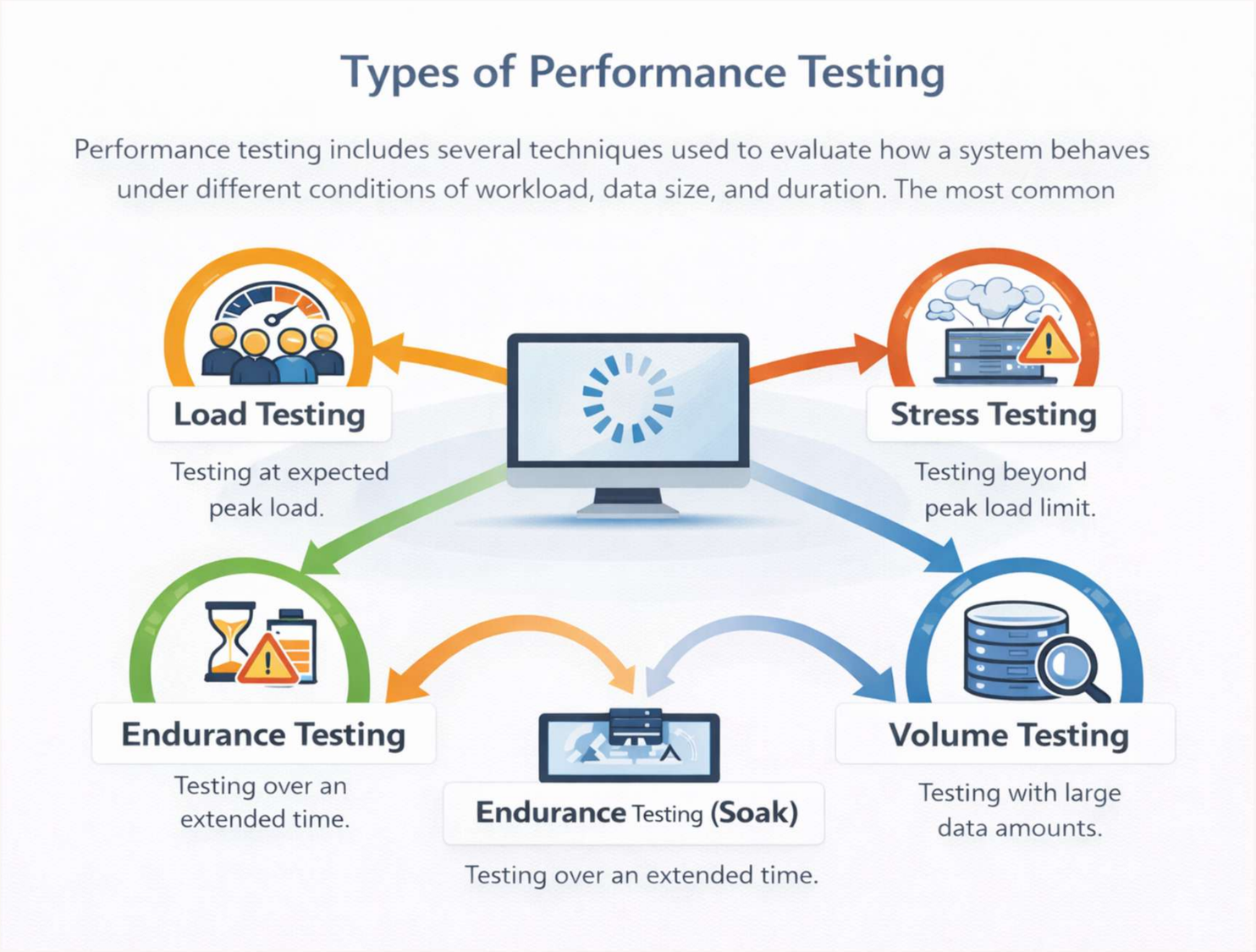
- Requests per second
- Transactions per second (TPS)



Resource Utilization

Resource utilization measures how much system resources (CPU, memory, disk, network) are being used while processing requests. Efficient systems should handle workloads using minimal resources.

Types of Performance Testing



Performance testing includes several techniques used to evaluate how a system behaves under different conditions of workload, data size, and duration. The most common types are:

- **Load Testing**
- **Stress Testing**
- **Endurance (Soak) Testing**
- **Volume (Flood) Testing**

LOAD TESTING

Load Testing

Load testing evaluates the system when it operates under the **expected or normal workload conditions.** The workload is usually defined in the **business requirements or Service Level Agreement (SLA)**. The system is tested with the exact number of users or transactions it is designed to support.

In simple terms, load testing answers the question:

“Can the system handle the expected number of users efficiently?”

- Load testing focuses on determining whether the system meets its SLA (Service Level Agreement).
- An SLA specifies acceptable performance levels such as: **Response time, Maximum number of users, System availability, Transaction processing time.**

Example

Consider an online retail website.

Business requirement(SLA):

- The website should support 1,000 concurrent users
- Product search results must appear within 3 seconds

During load testing:

- Testing tools simulate 1,000 users browsing products, adding items to carts, and making purchases simultaneously.
- Testers measure response time, throughput, and system stability.

If the response time stays below 3 seconds, the system successfully meets its SLA.
Tools such as **Apache JMeter** are often used to simulate these virtual users.

STRESS TESTING

Stress Testing

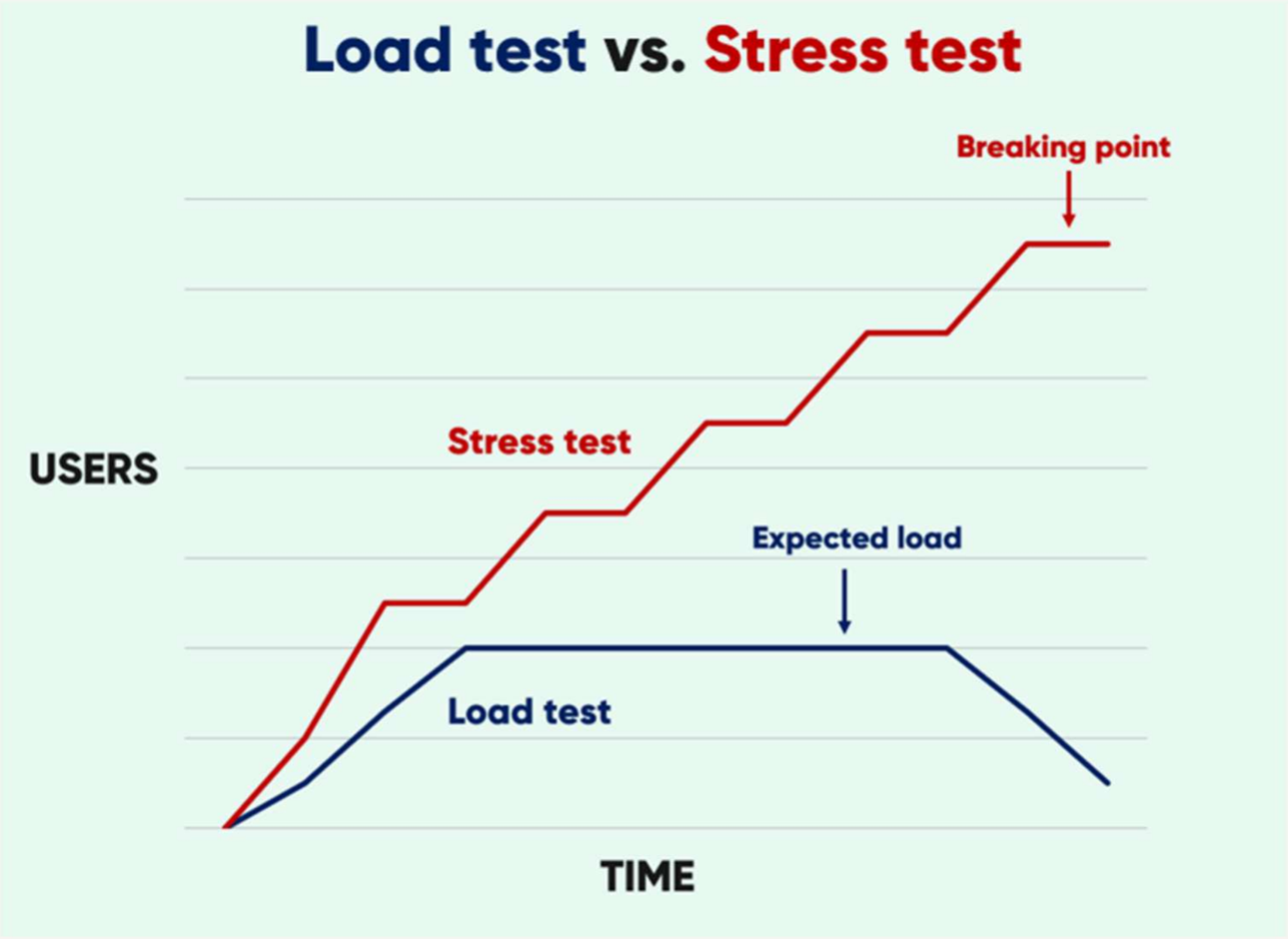
Stress testing evaluates the system by **pushing it beyond its normal operating capacity.**
The goal is to **determine the maximum load the system can handle before it fails.**

Stress testing answers the question:

"What happens when the system is overloaded?"

Stress testing focuses on:

- **System robustness**
- **Failure behavior**
- **System recovery**



When a system is overloaded, two possible outcomes may occur.

Graceful Degradation

- The system performance gradually slows down.
- The system continues to function but with reduced speed.

- Example:
  - Slower response times
  - Temporary error messages
  - Displaying a “Service Busy” message

- This is considered a desirable behavior because the system still remains operational.

Catastrophic Failure

- The system crashes completely.
- The database becomes locked.
- Data loss may occur.
- The application becomes unavailable.

- This indicates poor system design or inadequate infrastructure.

Example

Consider an airline ticket booking website.

Expected capacity:

- 1,000 concurrent users

During stress testing:

- The system is tested with 5,000 simultaneous users.

Possible observations:

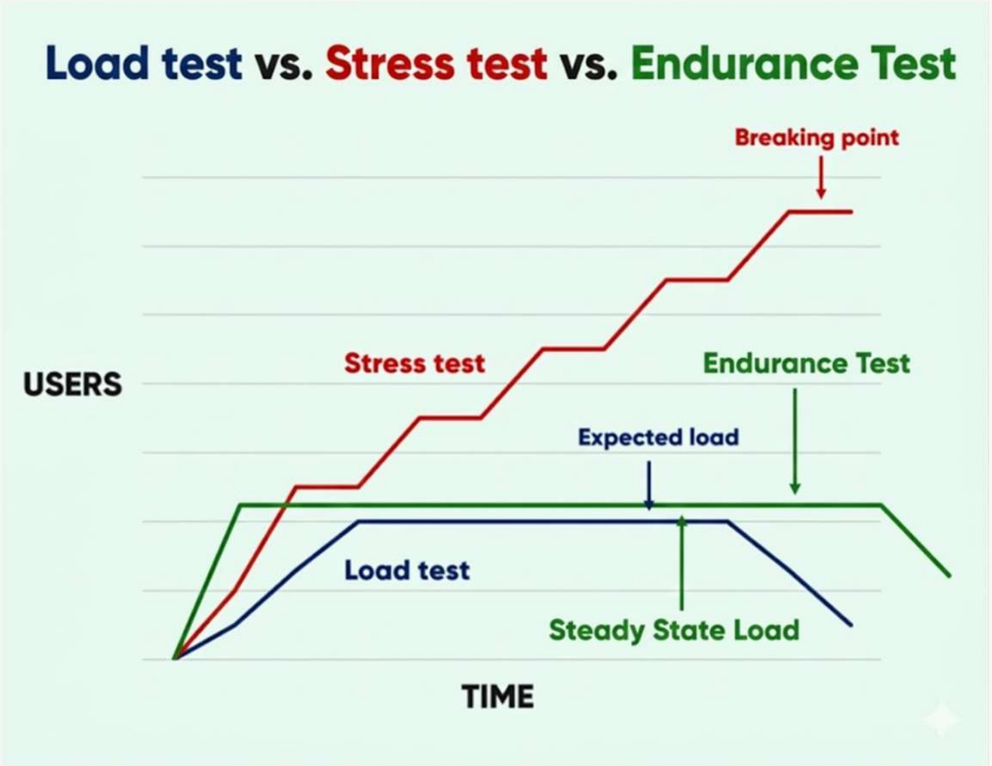
- At 2,500 users → response time increases.
- At 4,000 users → error messages appear.
- At 5,000 users → system crashes.
-

This test helps identify the breaking point of the system.

ENDURANCE (SOAK TESTING

Endurance (Soak) Testing

- Endurance testing evaluates system performance over a long duration while maintaining a steady load.
- The goal is to identify performance degradation or system failures that occur over time.
- Unlike load testing, which runs for short periods, endurance testing may run for 24 hours or 48 hours or 72 hours or Several days



"What happens to the system over a period of time?"

The main focus of endurance testing is detecting memory leaks and resource exhaustion.
A memory leak occurs when a program:

- Allocates memory for a task
- Fails to release that memory after completing the task

Over time, unused memory accumulates and eventually crashes the system.

Example

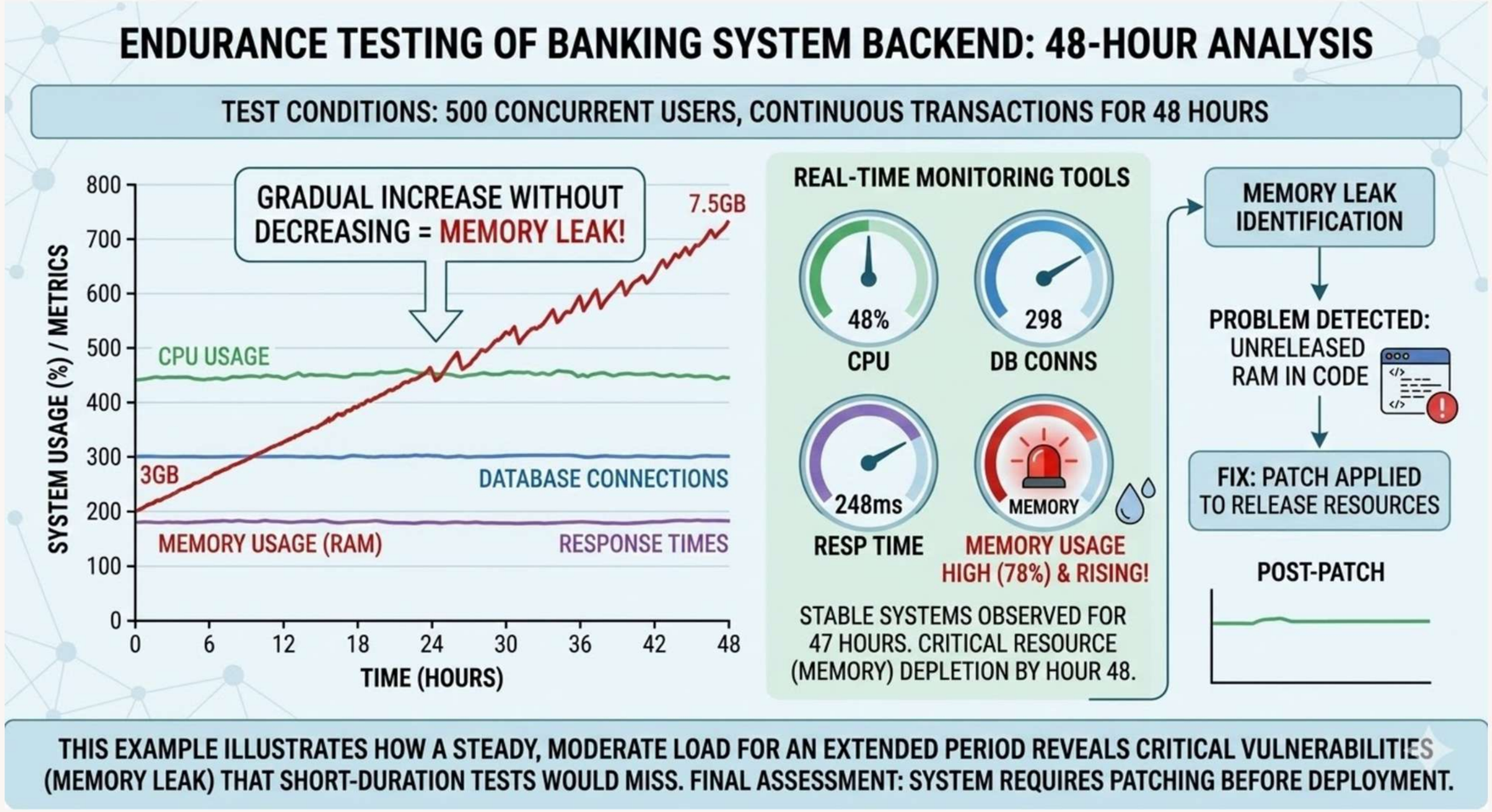
Consider a banking system backend.

Conditions during testing:

- 500 users performing transactions continuously
- The test runs for 48 hours

Monitoring tools observe:

- Memory usage
- CPU usage
- Database connections
- Response times



If memory usage gradually increases without decreasing, it indicates a memory leak.

A banking application must operate 24/7 without downtime. Endurance testing ensures that the system remains stable for long durations without requiring frequent restarts.

VOLUME (FLOOD) TESTING

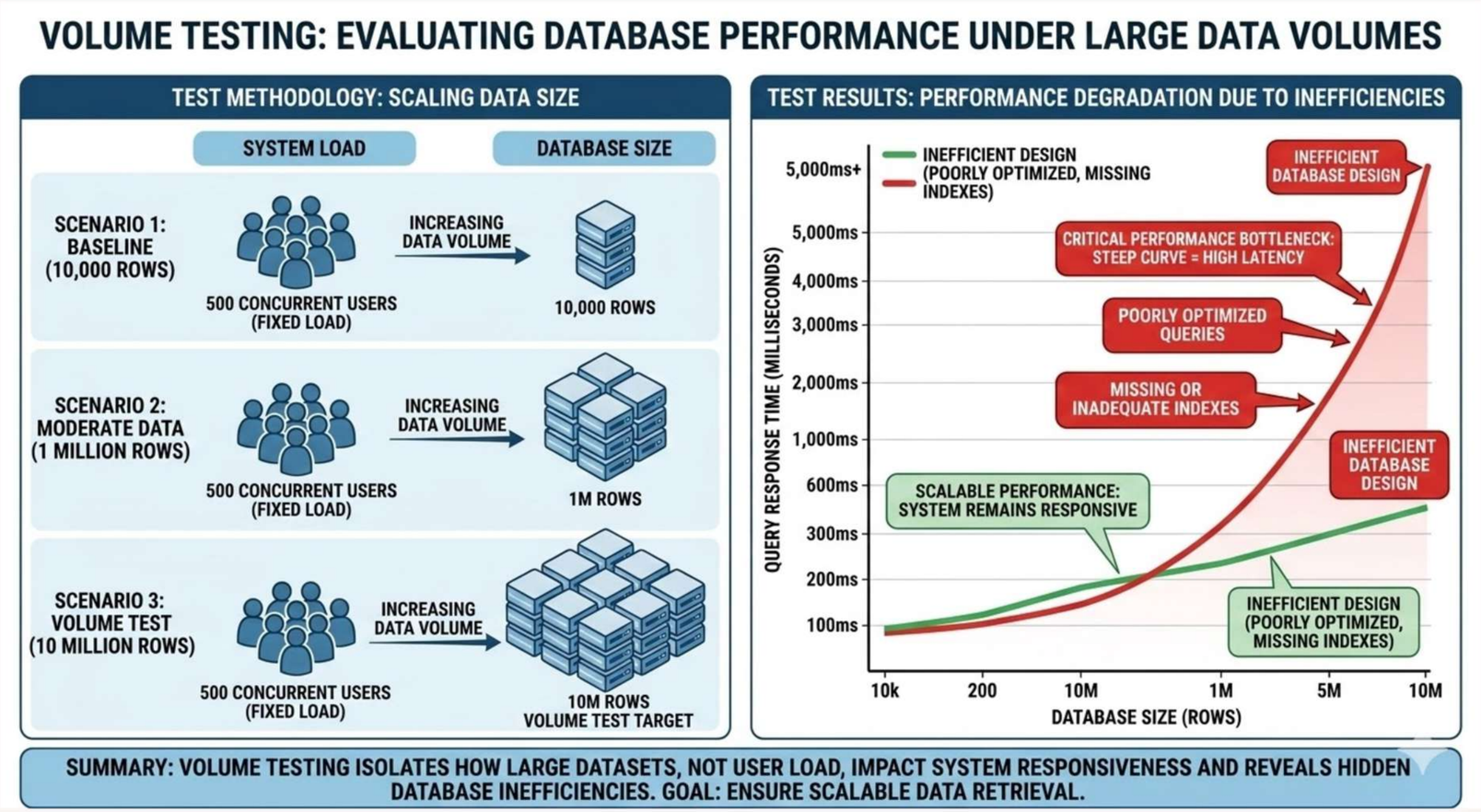
Volume (Flood) Testing

- Volume testing evaluates system performance when the database contains very large amounts of data.
- Instead of increasing the number of users, volume testing increases the size of the data stored in the system.
- This test checks whether the system can efficiently handle large data volumes.

Huge tables can make "Select" queries extremely slow if not properly indexed.

The main focus of volume testing is database efficiency and data retrieval performance.
Large datasets can slow down database operations if:

- Queries are poorly optimized
- Indexes are missing
- Database design is inefficient



Example

Consider a healthcare management system.

Initially, the database contains:

- 100,000 patient records

Over time, the system grows to store:

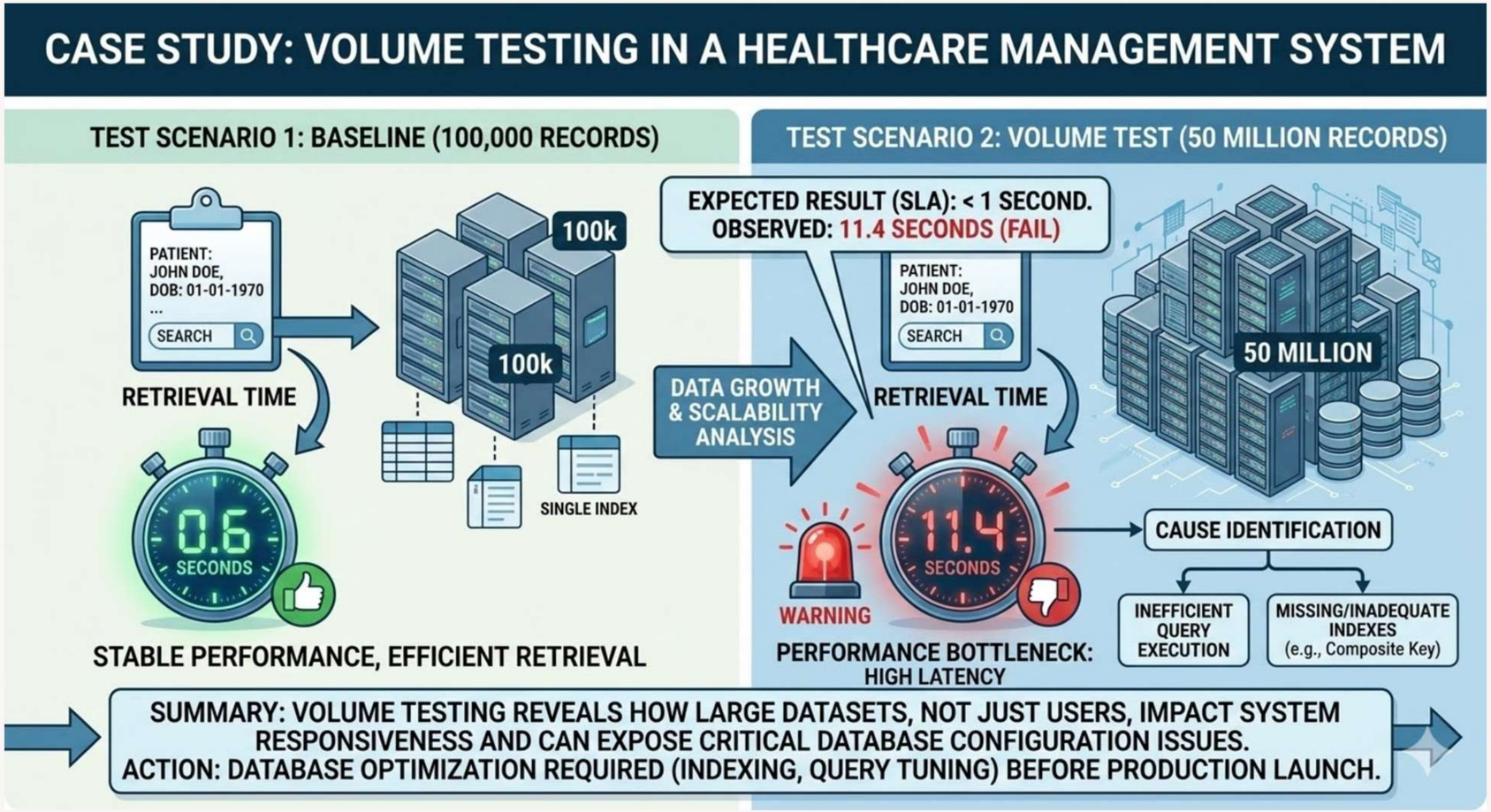
- 50 million patient records

During volume testing:

- The database is filled with millions of records.
- Testers perform operations like searching patient details.

Expected result:

- Patient record retrieval should occur within 1 second.



If it takes 10 seconds, database optimization may be required.

Large organizations such as hospitals and insurance companies store huge amounts of historical data. Volume testing ensures that database queries remain fast even when data size becomes very large.

Tools Used for Performance Testing

Performance testing tools simulate multiple users accessing a system simultaneously to measure its performance, including response time, throughput, and resource utilization.

Apache JMeter™

Open Source

- Popular & easy to use
- Supports web and API testing

LoadRunner

Commercial

- Enterprise-level testing
- Detailed analytics & reports

k6

Open Source

- DevOps-friendly
- JavaScript-based testing

LOCUST

Open Source

- Python-based scripting
- Easily scales distributed tests

SECURITY TESTING

SECURITY TESTING

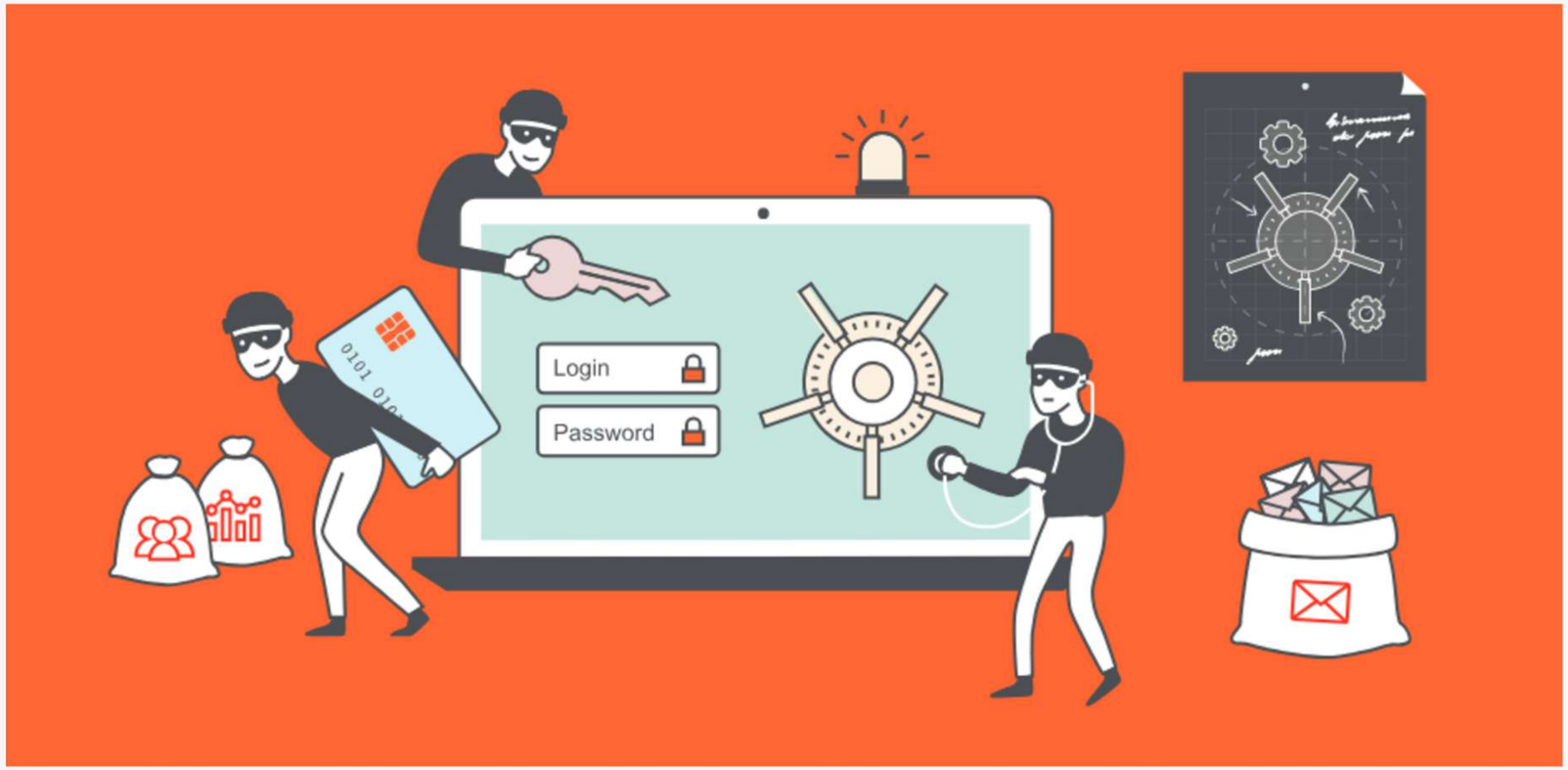


Image Source: PortSwigger

Security Testing

Security Testing is a type of Software Testing that uncovers vulnerabilities in the system and determines that the data and resources of the system are protected from possible intruders.

While performance testing asks **“Can the system handle load?”**, security testing asks “**Can the system defend itself from attackers?”**

- It ensures that the software system and application are free from any threats or risks that can cause a loss.
- Security testing of any system is focused on finding all possible loopholes and weaknesses of the system that might result in the loss of information of the organization.

Objectives of Security Testing:

- Protects sensitive information
- Prevents unauthorized access
- Maintains data integrity
- Ensures system availability
- Detects vulnerabilities and security loopholes

Software Security Fundamentals

Security testing follows seven core principles, often referred to as the **CIA triad (Confidentiality, Integrity, Availability)** with seven basic principles of security testing:

CIA Triad

1. **Confidentiality:** verifies that sensitive data is only accessible to authorized users, often through encryption and access control mechanisms. *For example, when a user enters their credit card details on Amazon or logs into their account on PayPal, the information is protected using HTTPS encryption.*
2. **Integrity:** Verifies that data remains unchanged and unaltered during storage or transmission. Hash functions and checksums are commonly used to guarantee integrity. Example, *if a hacker modifies a bank transaction amount from $100 to $10,000, integrity has been compromised.*
3. **Availability:** Ensures that the system remains functional and accessible, even under heavy traffic or during a cyberattack, such as a Distributed Denial of Service (DDoS) attack. Example, *Websites like Netflix and Google must remain available 24/7. A DDoS attack floods the server with traffic, causing the website to crash.*
4. **Authentication**: Ensures that only authorized users can access the system. This involves testing password policies, multi-factor authentication (MFA), and identity verification mechanisms. *Example, many modern platforms such as Google and Microsoft use multi-factor authentication to improve security.*
5. **Authorization**: Verifies that authenticated users can only access the resources and data they are authorized to use, through mechanisms such as role-based or attribute-based access control (RBAC and ABAC). *Example, If a patient tries to edit another patient's medical record, the system should deny access.*
6. **Non-Repudiation**: Ensures that users cannot deny their actions in the system. Digital signatures, audit logs, and transaction records are commonly used to guarantee non-repudiation. *Example, If the user claims: "I did not transfer this money."; The bank can check the transaction logs and digital signature to verify that the user performed the transaction.*
7. **Resilience**: Verifies the system's ability to recover from incidents, such as system crashes or attacks, by evaluating backup systems and response protocols. *Example, Consider a cloud-based streaming service such as Netflix. If a server fails due to heavy traffic: Backup servers automatically take over. Users continue streaming without interruption. This demonstrates system resilience.*

THREAT MODELING

THREAT MODELING

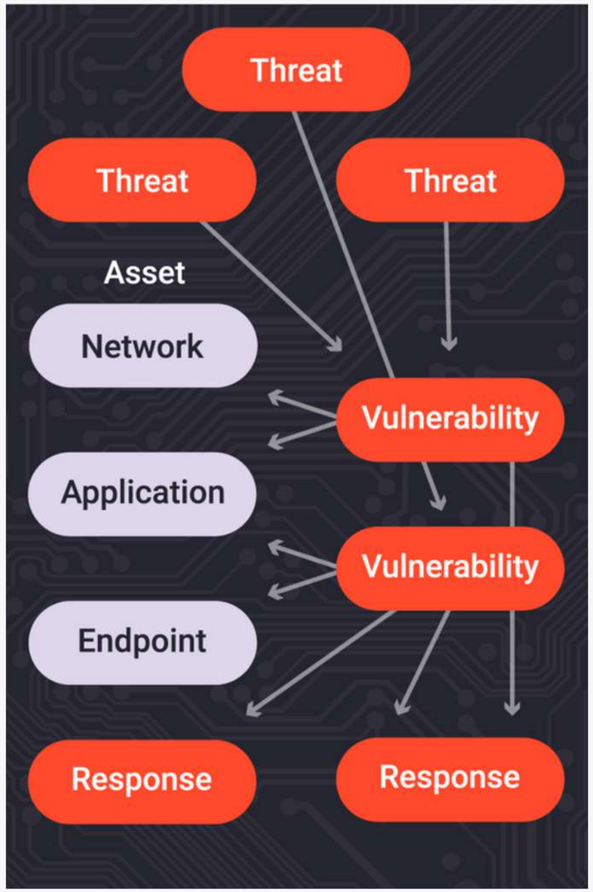


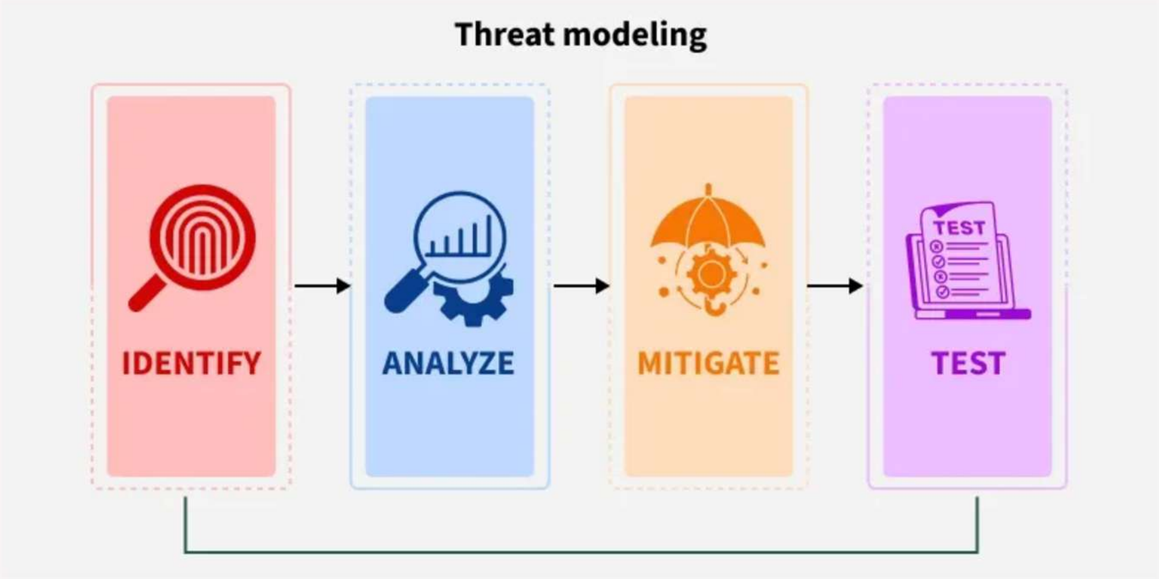
Image Source: G2 (Google Images)

Threat Modelling

- Threat modelling is a structured method to identify, analyze, and mitigate potential threats in systems, applications, or organizations.
- Threat modelling helps identify possible security threats during the design phase before development starts.
- It helps teams uncover vulnerabilities early and build security into the design before attackers exploit weaknesses.
- Identifies what can go wrong before deployment
- Helps understand attacker goals, techniques, and entry points
- Prioritises risks based on impact and likelihood
- Ensures security becomes part of the development lifecycle

Objectives of Security Testing:

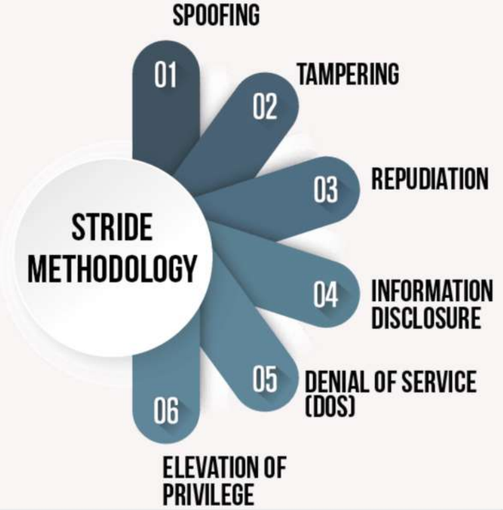
- Protects sensitive information
- Prevents unauthorized access
- Maintains data integrity
- Ensures system availability
- Detects vulnerabilities and security loopholes



Threat Modelling Method

STRIDE Model

The STRIDE model, created by Microsoft, categorizes six common security threats.



Type	Description	Security Property Affected	Example
Spoofing	Pretending to be another user or system to gain unauthorized access.	Authentication	An attacker steals login credentials and logs into someone else's bank account.
Tampering	Unauthorized modification of data in storage or during transmission.	Integrity	A hacker changes the price of a product in an e-commerce database from $500 to $5.
Repudiation	A user denies performing an action without proof to verify it.	Non-Repudiation	A user transfers money online and later claims they never made the transaction.
Information Disclosure	Sensitive information is exposed to unauthorized users.	Confidentiality	A website accidentally exposes user emails and passwords through an API.
Denial of Service (DoS)	Overloading a system so legitimate users cannot access it.	Availability	Attackers send millions of requests to crash a website server.
Elevation of Privilege	A user gains higher permissions than they are allowed.	Authorization	A normal user exploits a vulnerability and gains admin access to the system.

OWASP TOP 10

OWASP TOP 10

OWASP

TOP 10

Image Source: owasp.org

Introduction to OWASP

The **Open Web Application Security Project (OWASP)** tracks the most critical web risks.

OWASP Top 10 (2025)

The OWASP Top 10 is a list of the most critical security risks in web applications.

Why OWASP Top 10 is Important?

- Helps developers build secure applications
- Helps organizations identify common vulnerabilities
- Widely used as a security standard in industry
- Used in security testing and penetration testing

Rank	Vulnerability	Explanation
A01	Broken Access Control	Users can access resources or perform actions outside their permissions.
A02	Security Misconfiguration	Systems are incorrectly configured, exposing security weaknesses.
A03	Software Supply Chain Failures	Vulnerabilities introduced through third-party libraries, dependencies, or build tools.
A04	Cryptographic Failures	Sensitive data is not properly protected with encryption.
A05	Injection	Malicious input is executed as commands in the system.
A06	Insecure Design	Security flaws caused by poor system design.
A07	Authentication Failures	Weak identity verification mechanisms.
A08	Software and Data Integrity Failures	Systems fail to verify the integrity of software updates or data.
A09	Logging and Monitoring Failures	Lack of logging and monitoring prevents detection of attacks.
A10	Mishandling of Exceptional Conditions	Systems fail insecurely when unexpected errors occur.

A01 : Broken Access Control

Broken access control occurs when users can access resources or perform actions beyond their intended permissions. Access control ensures users only access data and functions they are authorized to use.

Prevention

- Role-Based Access Control (RBAC)
- Server-side authorization checks
- Principle of least privilege

A student portal allows a student to change:

```
portal.com/student/101
```

to

```
portal.com/student/102
```

and see another student's grades.

A02: Security Misconfiguration

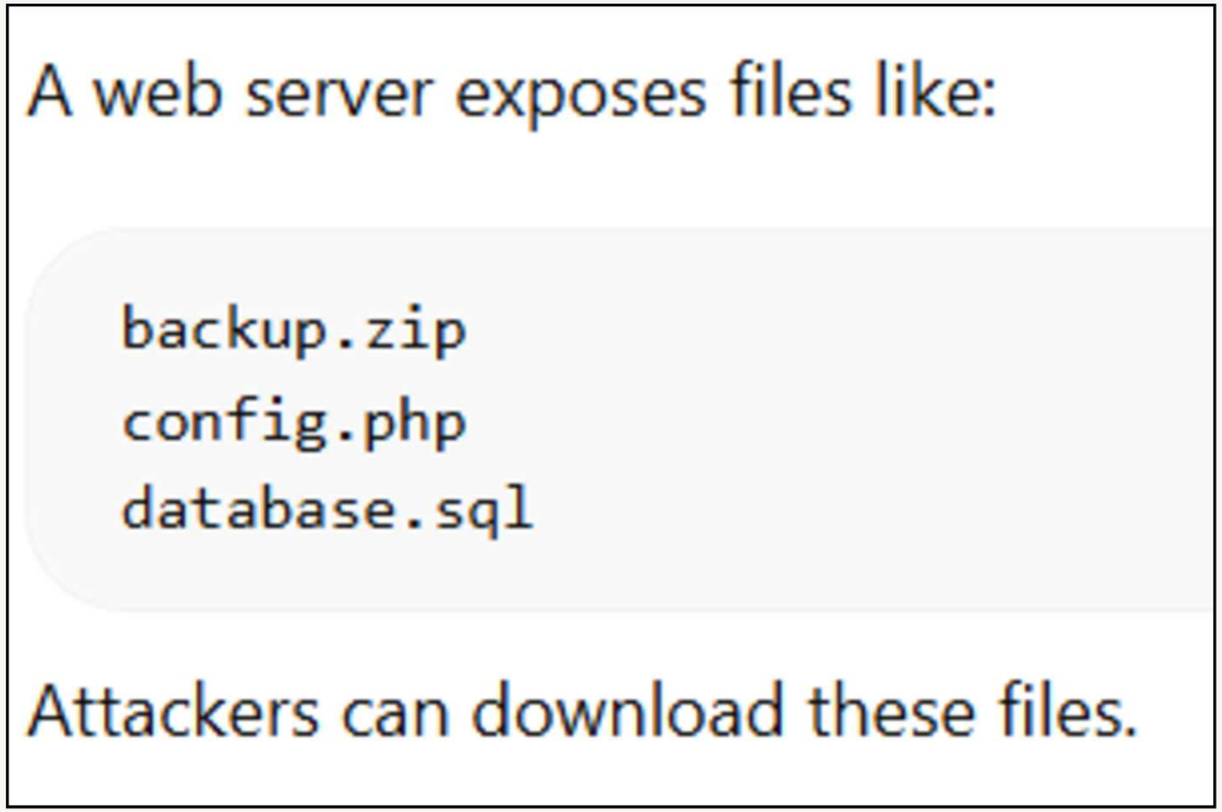
Security misconfiguration occurs when servers, frameworks, databases, or applications are incorrectly configured. This is one of the most common causes of security breaches.

Examples

- Default passwords are not changed
- Debug mode enabled in production
- Unnecessary services running
- Directory listing enabled

Prevention

- Secure default configurations
- Disable unnecessary features
- Regular security audits



A03: Software Supply Chain Failures

Occurs when vulnerabilities enter the system through third-party libraries, open-source dependencies, or build pipelines. Modern applications rely heavily on external packages and frameworks, which may contain vulnerabilities such as a malicious update is inserted into a dependency used by thousands of applications.

Example
A web application uses a vulnerable version of a JavaScript library.
If attackers exploit that library, they can attack the entire application.

Prevention

- Regular dependency updates
- Use trusted repositories
- Verify software packages
- Scan dependencies for vulnerabilities

A04: Cryptographic Failures

Cryptographic failures occur when sensitive information is not properly encrypted or protected.

Sensitive data includes:

- Passwords
- Credit card numbers
- Personal data
- Medical records

Prevention:

- Use strong encryption
- Secure password hashing
- Use HTTPS for data transmission

A database stores passwords like this:

```
Username: Alice
Password: password123
```

If attackers access the database, they immediately see the password.

A05: Injection

Injection occurs when untrusted input is interpreted as commands by the system. Attackers insert malicious code into input fields.

Common types

- SQL Injection
- Command Injection
- LDAP Injection

Prevention

- Input validation
- Parameterized queries
- Prepared statements

A login query:

```
SELECT * FROM users
WHERE username='admin'
AND password='password'
```

If an attacker enters:

```
' OR 1=1 --
```

The condition becomes true and login may succeed.

A06: Insecure Design

Insecure design happens when security is not considered during system design.
Even if code is correct, the architecture itself may be vulnerable.

Example
A banking system allows unlimited password attempts.
Attackers can run brute-force attacks to guess passwords.

Prevention

- Threat modeling
- Secure architecture design
- Security reviews during development

A07: Authentication Failures

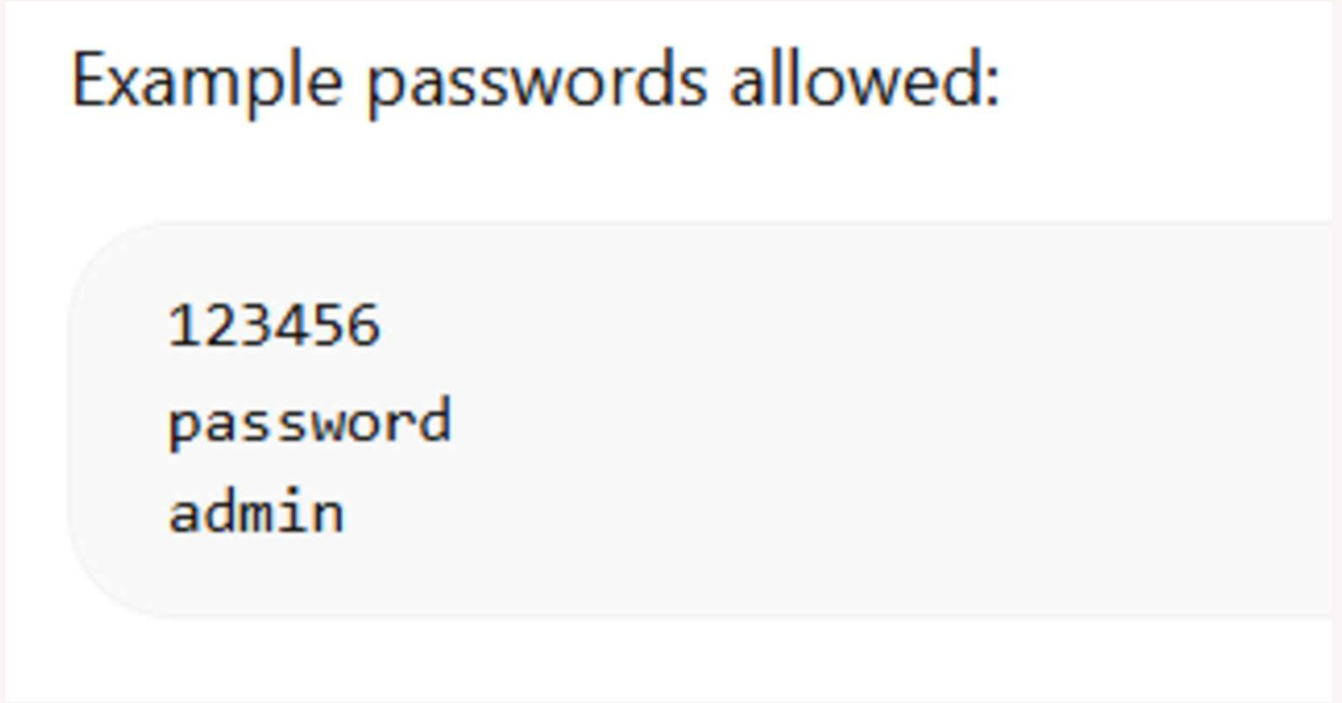
Occurs when systems fail to properly verify user identity. Authentication ensures the user is who they claim to be.

Examples

- Weak password policies
- No multi-factor authentication
- Session IDs exposed in URLs

Prevention

- Strong password requirements
- Multi-factor authentication (MFA)
- Secure session management



A08: Software and Data Integrity Failures

Occurs when systems do not verify the integrity of software updates or data. Attackers can modify or inject malicious code.

Example
An application downloads software updates automatically without verifying their authenticity.
An attacker inserts a malicious update.

Prevention

- Digital signatures
- Integrity checks
- Secure CI/CD pipelines

A09: Logging and Monitoring Failures

Occurs when systems do not properly log or monitor security events. Without logs, organizations cannot detect attacks.

Example
An attacker attempts 1000 failed login attempts.
If the system does not record or alert this activity, the attack continues unnoticed.

Prevention

- Enable logging
- Monitor suspicious activities
- Use intrusion detection systems

A10: Mishandling of Exceptional Conditions

Occurs when systems handle errors or unexpected conditions insecurely. Systems should fail safely, not expose sensitive information.

Prevention

- Secure error handling
- Do not expose internal system information
- Implement safe failure mechanisms

An error message reveals internal system details:

```
Database connection failed: password=db123
```

Attackers can use this information to attack the system.

PENETRATION TESTING

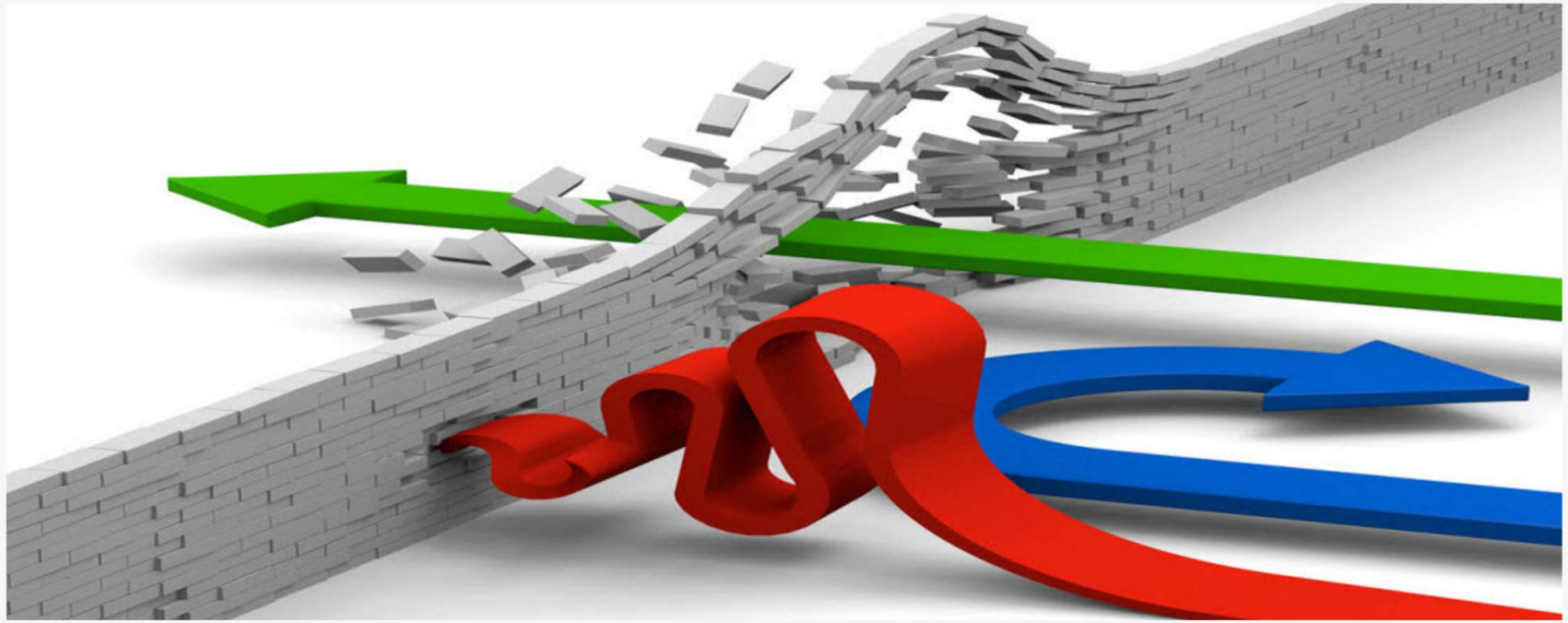


Image Source: Mindcentric(Google Images)

Penetration Testing

Penetration testing is a controlled and authorized cyberattack performed to **identify security vulnerabilities in a system, network, or application.** The goal is not to damage the system, but to discover weaknesses before malicious attackers do.

Pen-testing is commonly used by organizations to evaluate the security of:

- Web applications
- Networks
- Servers
- Mobile applications
- Cloud systems

Professional security testers (ethical hackers) simulate real-world attacks to determine how secure the system is.

Penetration Testing Process

Pen-testing typically follows five main phases:

Reconnaissance (Information Gathering)
Reconnaissance is the first stage of penetration testing, where testers gather as much information as possible about the target system.

Scanning

Scanning involves actively probing the system to identify open ports, services, and vulnerabilities.

Gaining Access (Exploitation)

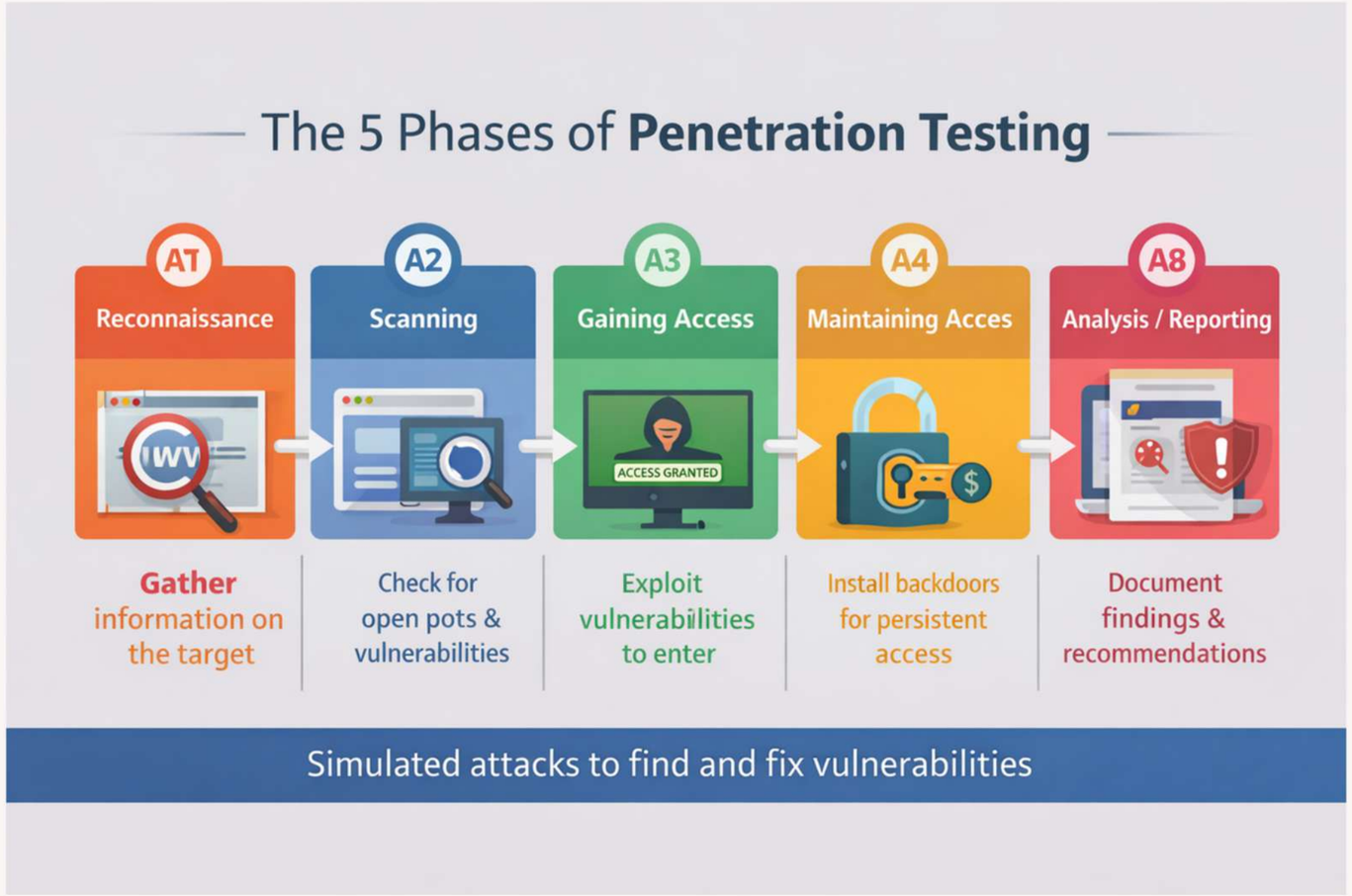
In this phase, the tester attempts to exploit discovered vulnerabilities to gain access to the system.

Maintaining Access

Once access is gained, the tester attempts to maintain access to the system over time.

Analysis and Reporting

The final phase involves documenting the findings of the penetration test.



Penetration Testing Process

Pen-testing typically follows five main phases:

Reconnaissance (Information Gathering)

Reconnaissance is the first stage of penetration testing, where testers gather as much information as possible about the target system. This stage is similar to a thief studying a building before attempting a break-in. The more information collected, the easier it is to find weaknesses.

Types of Reconnaissance:

Passive Reconnaissance

- Information gathered without interacting with the target system
- No direct connection with the server

Examples:

- Searching company websites
- Social media research
- Public databases
- WHOIS domain information

Active Reconnaissance

- Direct interaction with the system
- The target may detect these activities

Examples:

- Ping scans
- Port scans
- Network probing

Information Collected

Pen-testers may collect:

- IP addresses
- Domain names
- Email addresses
- Employee information
- Network infrastructure
- Technology stack (server software, frameworks)

Example:

A tester may discover the company website runs on Apache server with PHP, which may contain known vulnerabilities.

Penetration Testing Process

Pen-testing typically follows five main phases:

Scanning

In this phase, various tools and techniques are used to understand how the target application responds to intrusion attempts. **Static analysis** involves inspecting the application's code without executing it. This helps identify potential vulnerabilities based on code structure and logic. **Dynamic analysis** involves inspecting the application's behavior while it's running. This provides real-time insights into how the application responds to different inputs and interactions.
Scanning involves actively probing the system to identify open ports, services, and vulnerabilities. A port is a communication endpoint used by applications. If a port is open, attackers may attempt to exploit the service running on it.

Common Activities in Scanning

- Network scanning
- Vulnerability scanning
- Service detection
- Operating system detection

A tester scans a server and finds:

Port	Service	Status
22	SSH	Open
80	Web Server	Open
3306	MySQL	Open

An open database port (3306) may indicate **a potential vulnerability** if it is publicly accessible.

Penetration Testing Process

Pen-testing typically follows five main phases:

Gaining Access (Exploitation)

In this phase, the tester attempts to exploit discovered vulnerabilities to gain access to the system.

This simulates what a real attacker would do after identifying weaknesses.

The tester may attempt to exploit vulnerabilities such as:

- SQL Injection
- Weak passwords
- Outdated software
- Misconfigured services

If a web application has a **SQL Injection vulnerability**, a tester might manipulate the login input.

Example login query:

```
SELECT * FROM users
WHERE username='admin'
AND password='password'
```

If the attacker enters:

```
' OR 1=1 --
```

The condition becomes true, and the system may grant unauthorized access.

This allows the tester to **log into the system without valid credentials.**

Penetration Testing Process

Pen-testing typically follows five main phases:

Maintaining Access

- Once access is gained, the tester attempts to maintain access to the system over time.
- Real attackers do this to continue stealing data or controlling the system.
- This stage evaluates whether attackers could remain undetected for long periods.

Techniques Used

Attackers may install:

- Backdoors
- Hidden user accounts
- Malware
- Remote access tools

A backdoor allows attackers to re-enter the system without repeating the original attack.

An attacker creates a hidden administrator account:

```
username: sys_admin_backup
password: ********
```

Even if the system password is changed, the attacker can still log in through the hidden account.

Purpose of This Phase

Pen-testers evaluate:

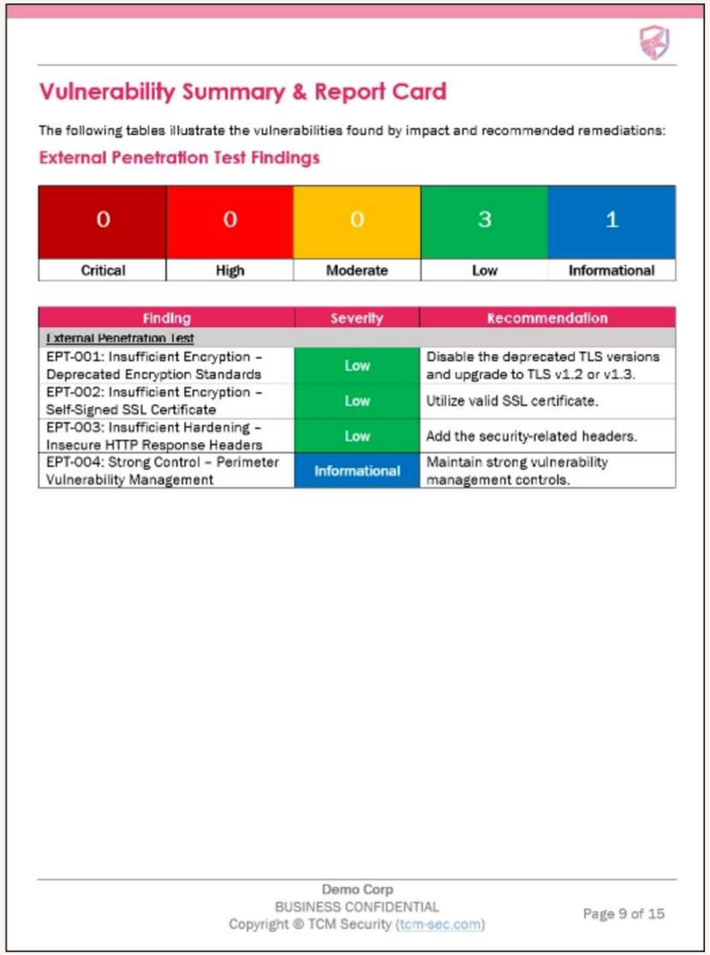
- How long attackers could stay in the system
- Whether the system detects unauthorized activity
- Whether security monitoring tools identify suspicious behavior

Penetration Testing Process

Pen-testing typically follows five main phases:

Analysis and Reporting

- The final phase involves documenting the findings of the penetration test.
- This is one of the most important phases, because the goal of pen-testing is improving security, not just breaking systems.
- This report includes information about the vulnerabilities that were successfully exploited, any sensitive data accessed, and the duration of undetected access.
- Security personnel analyze this information to understand the effectiveness of existing security measures and to prioritize remediation efforts.



Vulnerability Summary & Report Card

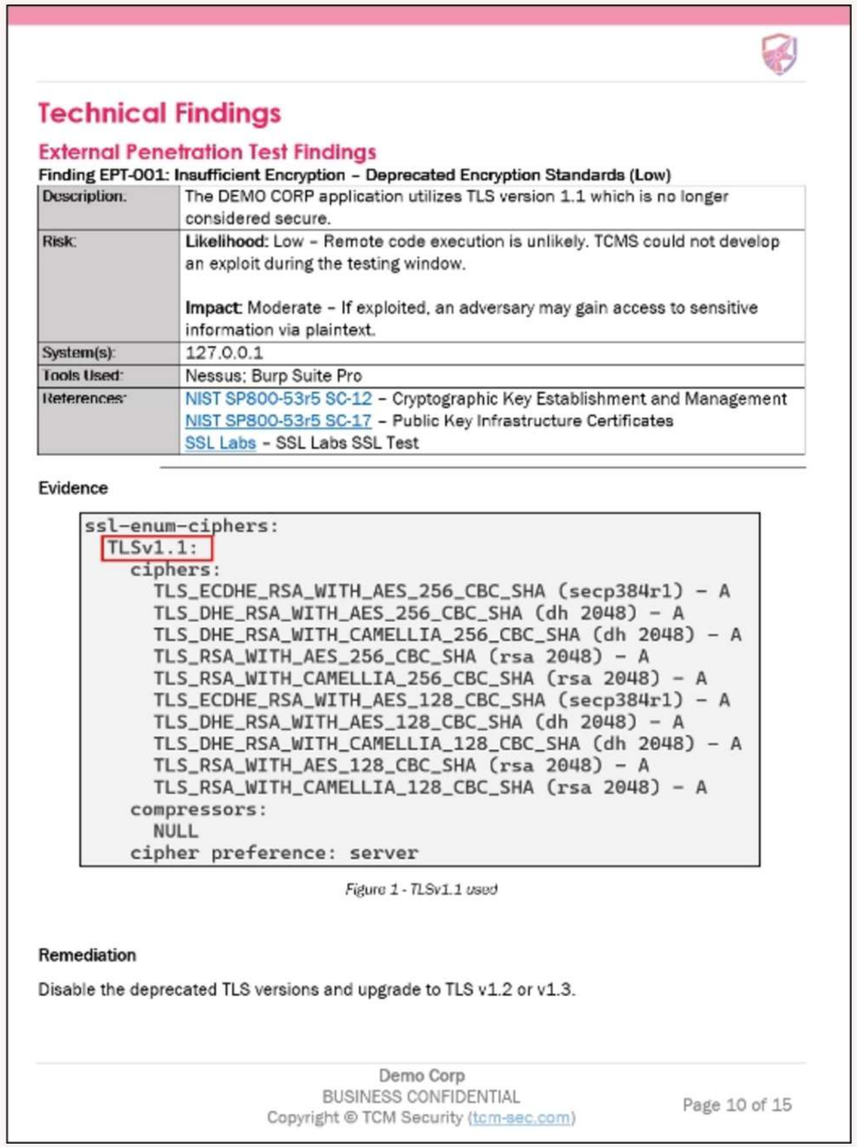
The following tables illustrate the vulnerabilities found by impact and recommended remediations:

External Penetration Test Findings

0	0	0	3	1
Critical	High	Moderate	Low	Informational

Finding	Severity	Recommendation
External Penetration Test		
EPT-001: Insufficient Encryption – Deprecated Encryption Standards	Low	Disable the deprecated TLS versions and upgrade to TLS v1.2 or v1.3.
EPT-002: Insufficient Encryption – Self-Signed SSL Certificate	Low	Utilize valid SSL certificate.
EPT-003: Insufficient Hardening – Insecure HTTP Response Headers	Low	Add the security-related headers.
EPT-004: Strong Control – Perimeter Vulnerability Management	Informational	Maintain strong vulnerability management controls.

Demo Corp
BUSINESS CONFIDENTIAL
Copyright © TCM Security (tcm-sec.com)
Page 9 of 15



Technical Findings

External Penetration Test Findings

Finding EPT-001: Insufficient Encryption – Deprecated Encryption Standards (Low)

Description:	The DEMO CORP application utilizes TLS version 1.1 which is no longer considered secure.
Risk:	**Likelihood:** Low – Remote code execution is unlikely. TCMS could not develop an exploit during the testing window. **Impact:** Moderate – If exploited, an adversary may gain access to sensitive information via plaintext.
System(s):	127.0.0.1
Tools Used:	Nessus; Burp Suite Pro
References:	NIST SP800-53r5 SC-12 – Cryptographic Key Establishment and Management NIST SP800-53r5 SC-17 – Public Key Infrastructure Certificates SSL Labs – SSL Labs SSL Test

Evidence

```
ssl-enum-ciphers:
  TLSv1.1:
    ciphers:
      TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (secp384r1) - A
      TLS_DHE_RSA_WITH_AES_256_CBC_SHA (dh 2048) - A
      TLS_DHE_RSA_WITH_CAMELLIA_256_CBC_SHA (dh 2048) - A
      TLS_RSA_WITH_AES_256_CBC_SHA (rsa 2048) - A
      TLS_RSA_WITH_CAMELLIA_256_CBC_SHA (rsa 2048) - A
      TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (secp384r1) - A
      TLS_DHE_RSA_WITH_AES_128_CBC_SHA (dh 2048) - A
      TLS_DHE_RSA_WITH_CAMELLIA_128_CBC_SHA (dh 2048) - A
      TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A
      TLS_RSA_WITH_CAMELLIA_128_CBC_SHA (rsa 2048) - A
    compressors:
      NULL
    cipher preference: server
```

Figure 1 - TLSv1.1 used

Remediation

Disable the deprecated TLS versions and upgrade to TLS v1.2 or v1.3.

Demo Corp
BUSINESS CONFIDENTIAL
Copyright © TCM Security (tcm-sec.com)
Page 10 of 15

Image Source: TCMSecurity(Google Images)

SCALABILITY TESTING IN CLOUD ENVIRONMENTS

SCALABILITY TESTING IN CLOUD ENVIRONMENTS

Image Source: Appquarterz(Google Images)

Cloud Computing

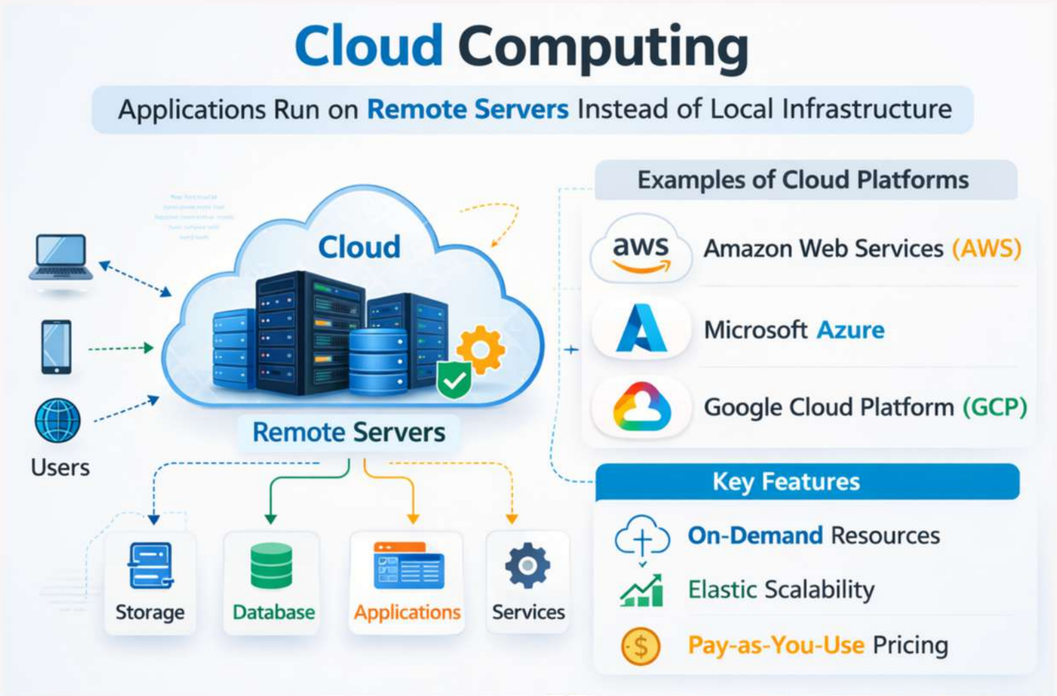
Cloud computing allows applications to run on remote servers instead of local infrastructure.

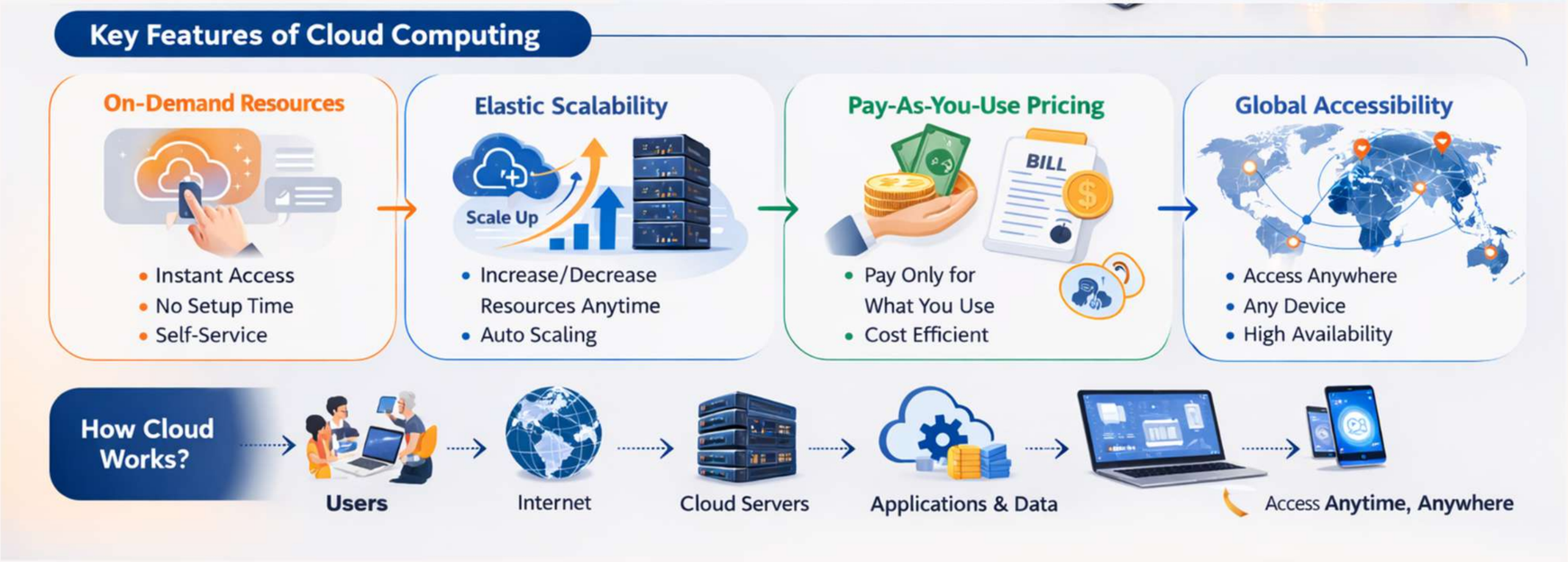
Examples of cloud platforms:

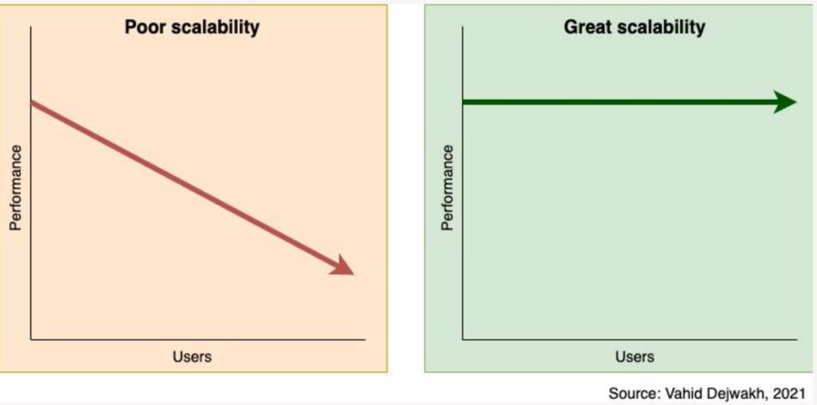
- Amazon Web Services
- Microsoft Azure
- Google Cloud Platform

Key features:

- On-demand resources
- Elastic scalability
- Pay-as-you-use pricing





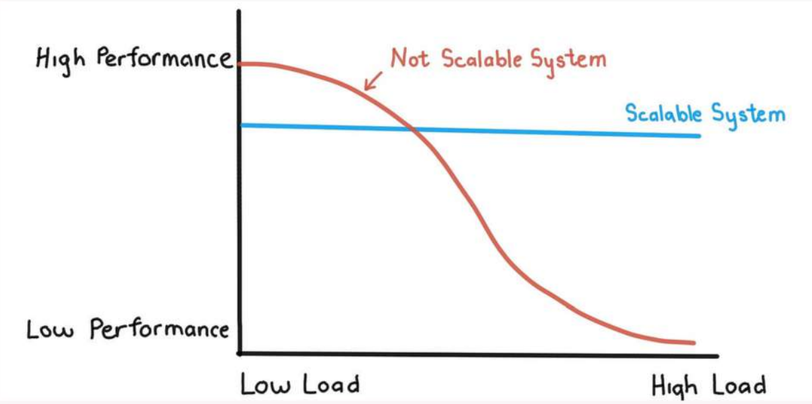


Source: Vahid Dejwakh, 2021

Scalability vs Performance

Performance
Speed of response under a specific workload

Scalability
Ability to maintain performance when workload increases

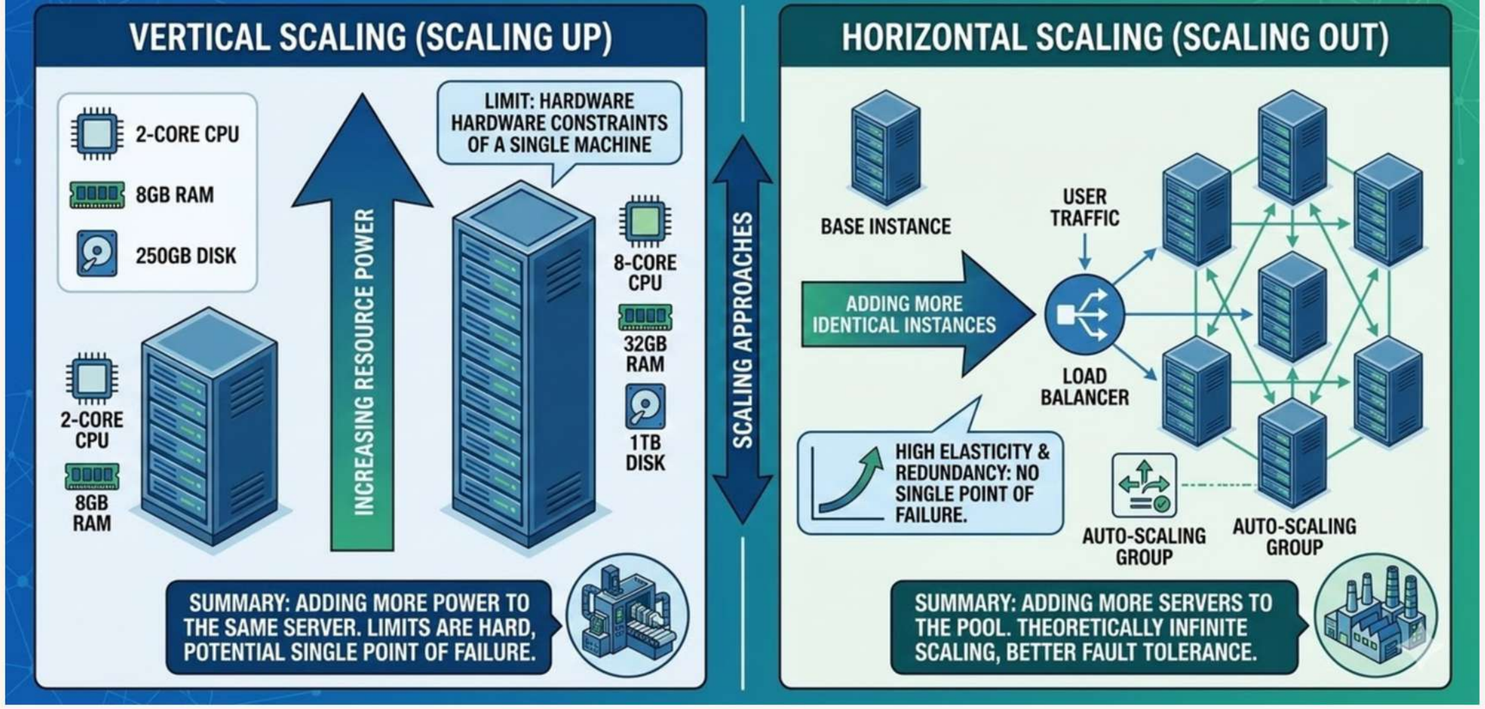


A system responding in 200ms with 100 users may degrade to 5 seconds with 10,000 users if scalability is poor.

Types of Scalability in Distributed Systems

Scale-up (vertical) = “make one machine stronger.”

Scale-out (horizontal) = “add more machines to share the work.”



Vertical Scalability (Scale-Up)

Increasing the resources of a single virtual machine (VM) or instance.

Examples:

- Upgrading an EC2 instance in AWS from t2.micro to m5.large (more CPU, RAM).
- Expanding storage capacity of a single database server.

Limitations:

- Even in the cloud, each VM type has maximum limits (e.g., CPUs, memory).
- Costs rise quickly with larger instance types.

Use case: Suitable for monolithic applications or databases that cannot easily be distributed.

Horizontal Scalability (Scale-Out)

Adding more instances/nodes to share the workload.

Examples:

- Deploying multiple web servers behind a load balancer (AWS ELB, Azure Load Balancer).
- Auto-scaling groups in AWS that spin up new instances when traffic spikes.

Advantages:

- Cloud makes this easy with elastic scaling- resources can be added or removed automatically.
- Works well for microservices and distributed systems.

Challenges:

- Requires careful design: stateless services, distributed databases, monitoring, and orchestration.
- Network latency and consistency issues must be managed.

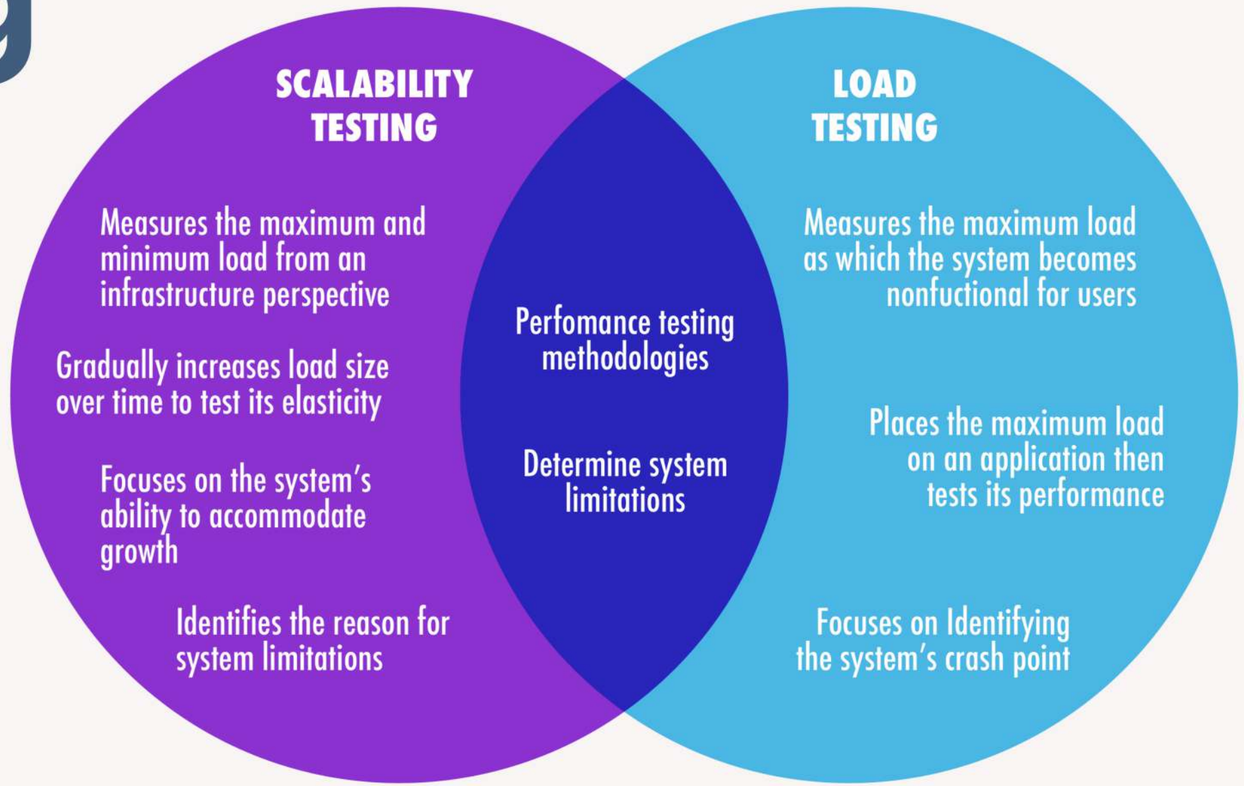
Scalability Testing

Scalability testing measures how well a system handles increasing workloads. The goal is to determine whether the system can scale up or down based on demand.

Cloud applications often experience sudden traffic spikes. *During Big billion days, e-commerce sites may experience 10x normal traffic.*

It focuses on:

- throughput
- response time
- resource utilization
- system stability



A website should handle:

- 100 users
- 1,000 users
- 100,000 users

without performance degradation.

Key Metrics in Scalability Testing

Metric	Description	Example Scenario
Response Time	The total time taken for a system to process a request and return a response to the user. It includes processing time, network delays, and database access time.	A user clicks "Search Products" on an e-commerce website and receives results in **300 ms**. If traffic increases and the response time rises to **4 seconds**, scalability issues may exist.
Throughput	The number of requests or transactions a system can process within a given time period (usually **requests per second (RPS)** or **transactions per second (TPS)**).	A payment system processes **1,500 transactions per second** during normal operation. If the system cannot handle more than **2,000 TPS**, that becomes the throughput limit.
CPU Usage	The percentage of processor capacity used by the system while handling workloads. High CPU usage may indicate computational bottlenecks or inefficient algorithms.	If a server reaches **95% CPU usage** when traffic reaches **3,000 users**, the system may not scale further without additional computing resources.
Memory Usage	The amount of RAM consumed by the application while processing requests. Memory leaks or inefficient memory management can degrade performance over time.	A microservice uses **1 GB RAM initially**, but gradually increases to **4 GB during heavy traffic**, indicating possible memory leaks.
Network Latency	The time delay involved in transmitting data between the client and server over the network. High latency affects user experience, especially in distributed systems.	A cloud application deployed in the US might have **20 ms latency locally**, but **200 ms latency for users in Asia**.

Steps in Conducting Scalability Testing

Scalability testing is a systematic process used to evaluate how well a system can handle increasing workloads. It involves defining performance metrics, identifying scalability factors, designing test scenarios, executing tests, and analyzing results.

1 Define the Performance Metrics

Performance metrics are quantitative indicators used to evaluate system performance under different workloads. Before performing scalability testing, it is necessary to determine which performance metrics will be measured during the tests.

Common metrics include:

Response Time: The time taken by the system to process a request and return a response to the user.

Throughput: The number of requests or transactions the system can process in a given time period.

Concurrency: The number of users or processes accessing the system simultaneously.

Resource Utilization: The amount of system resources used, such as CPU, memory, and network bandwidth.

Steps in Conducting Scalability Testing

Scalability testing is a systematic process used to evaluate how well a system can handle increasing workloads. It involves defining performance metrics, identifying scalability factors, designing test scenarios, executing tests, and analyzing results.

2. Identify the Scalability Factors

Scalability factors are variables that influence the system's ability to scale when workload increases. These factors determine how the system behaves under increasing demand.

Common scalability factors include:
Number of Users: Concurrent users accessing the system
Data Volume: Amount of data stored or processed
Transaction Rate: Number of requests sent per second
System Complexity: Number of services or components

For each factor, testers define:

- **Minimum workload (baseline)**
- **Expected workload**
- **Maximum workload (stress condition)**

Factor	Minimum	Expected	Maximum
Concurrent Users	100	2000	10000

This helps determine **the system's scalability limits.**

Steps in Conducting Scalability Testing

Scalability testing is a systematic process used to evaluate how well a system can handle increasing workloads. It involves defining performance metrics, identifying scalability factors, designing test scenarios, executing tests, and analyzing results.

3. Define the Test Scenarios

Test scenarios simulate realistic user behavior and workloads to evaluate system scalability. A good test scenario should represent actual usage patterns of the application.

Example:
For an online shopping platform:

- **Scenario 1:** 1000 users browsing products
- **Scenario 2:** 3000 users browsing + 1000 users adding items to cart
- **Scenario 3:** 5000 users placing orders simultaneously

These scenarios help evaluate how the system performs under different demand levels.

Steps in Conducting Scalability Testing

Scalability testing is a systematic process used to evaluate how well a system can handle increasing workloads. It involves defining performance metrics, identifying scalability factors, designing test scenarios, executing tests, and analyzing results.

4. Prepare the Test Environment

The test environment must closely resemble the production environment to ensure accurate results.

Important elements include:

- Hardware configuration
- Server infrastructure
- Network bandwidth
- Database systems
- Application architecture

If the environment differs significantly from production, the test results may be inaccurate.

Component	Configuration
Application Server	4 CPU cores, 16GB RAM
Database	Distributed SQL database
Network	1 Gbps bandwidth
Load Testing Tool	Automated traffic generator

Steps in Conducting Scalability Testing

Scalability testing is a systematic process used to evaluate how well a system can handle increasing workloads. It involves defining performance metrics, identifying scalability factors, designing test scenarios, executing tests, and analyzing results.

5 Conduct the Scalability Tests

During this phase, testers execute the defined test scenarios using load testing tools and monitor system performance.

Common tools used for scalability testing include:

- Apache JMeter
- Gatling
- Locust

The tools simulate large numbers of concurrent users interacting with the system.

During the test, engineers monitor:

- response time
- throughput
- CPU utilization
- memory usage
- error rate

Steps in Conducting Scalability Testing

Scalability testing is a systematic process used to evaluate how well a system can handle increasing workloads. It involves defining performance metrics, identifying scalability factors, designing test scenarios, executing tests, and analyzing results.

6. Analyze the Test Results

After running the tests, the collected data must be analyzed to understand how the system behaves under increasing workloads.

The analysis focuses on:

- identifying performance bottlenecks
- determining system capacity
- evaluating scalability limits

Engineers may also determine the system saturation point, where performance begins to degrade significantly.

Example:

Throughput increases steadily until 3000 users, after which response time rises sharply.

This indicates that the system cannot scale efficiently beyond that point without architectural improvements.

Issue	Cause	Solution
Slow response time	Database query bottleneck	Optimize queries
High CPU usage	Inefficient processing	Improve algorithms
High error rate	Server overload	Add more servers

TESTING IN DEVOPS

TESTING IN DEVOPS

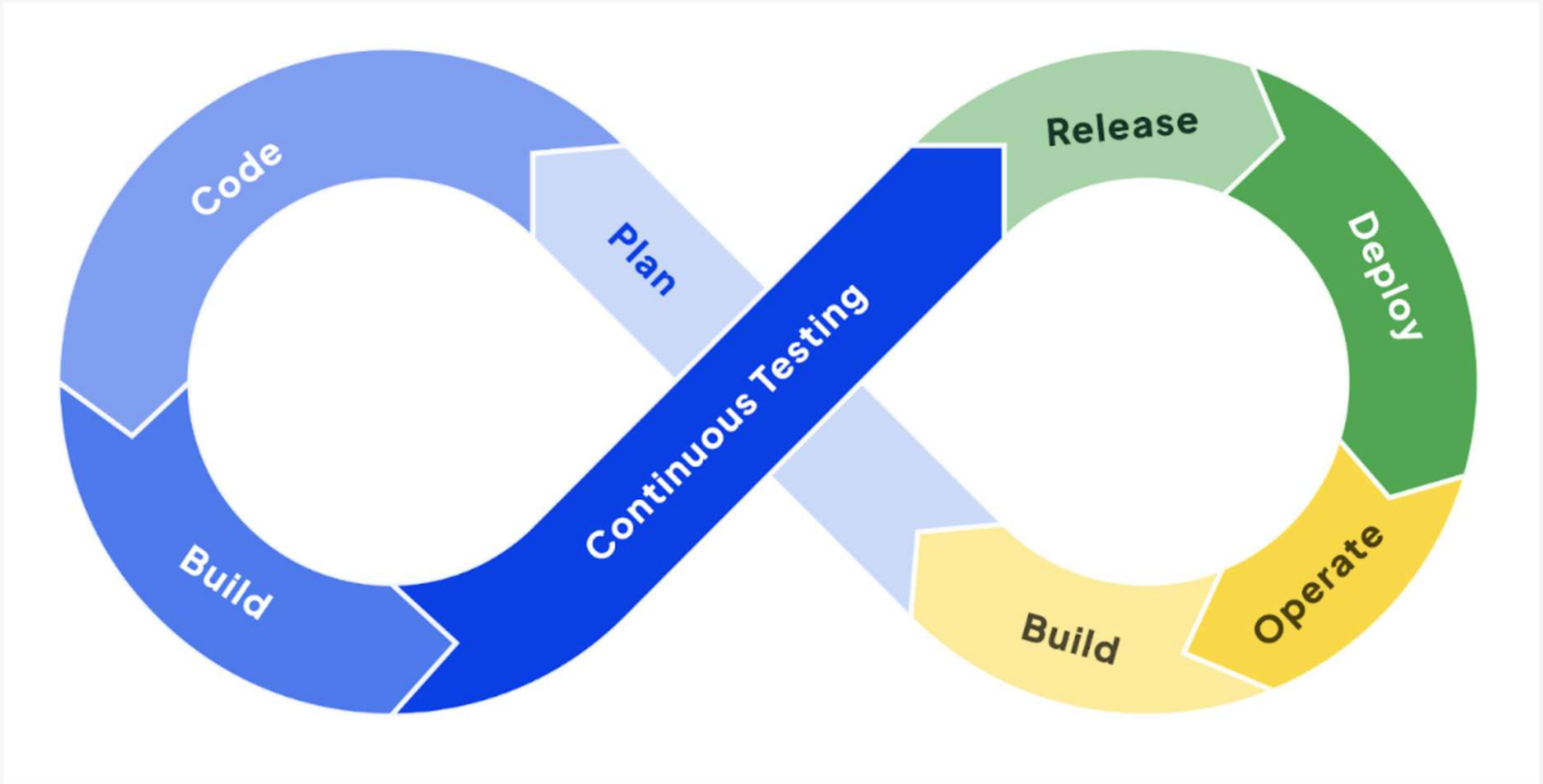


Image Source: Uffizzi(Google Images)

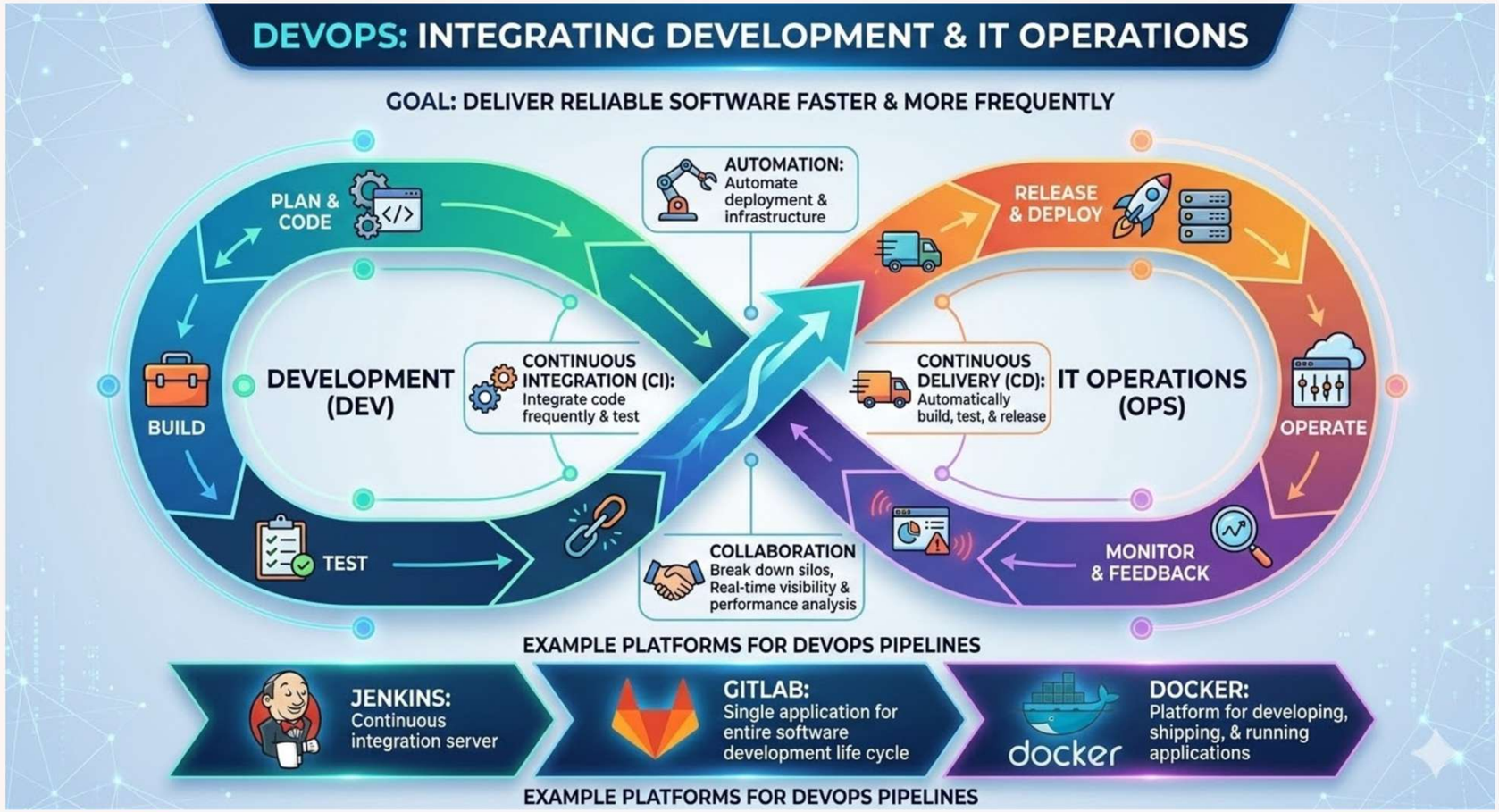
What is DevOps?

DevOps is a software engineering approach that integrates software development (Dev) and IT operations (Ops) to improve collaboration and automate the software delivery process. The goal is to deliver reliable software faster and more frequently.

DevOps focuses on:

- Automation
- Continuous integration
- Continuous delivery
- Collaboration between teams
- Continuous monitoring

Many organizations use platforms such as Jenkins, GitLab, Docker etc. to automate DevOps pipelines.



DevOps Lifecycle

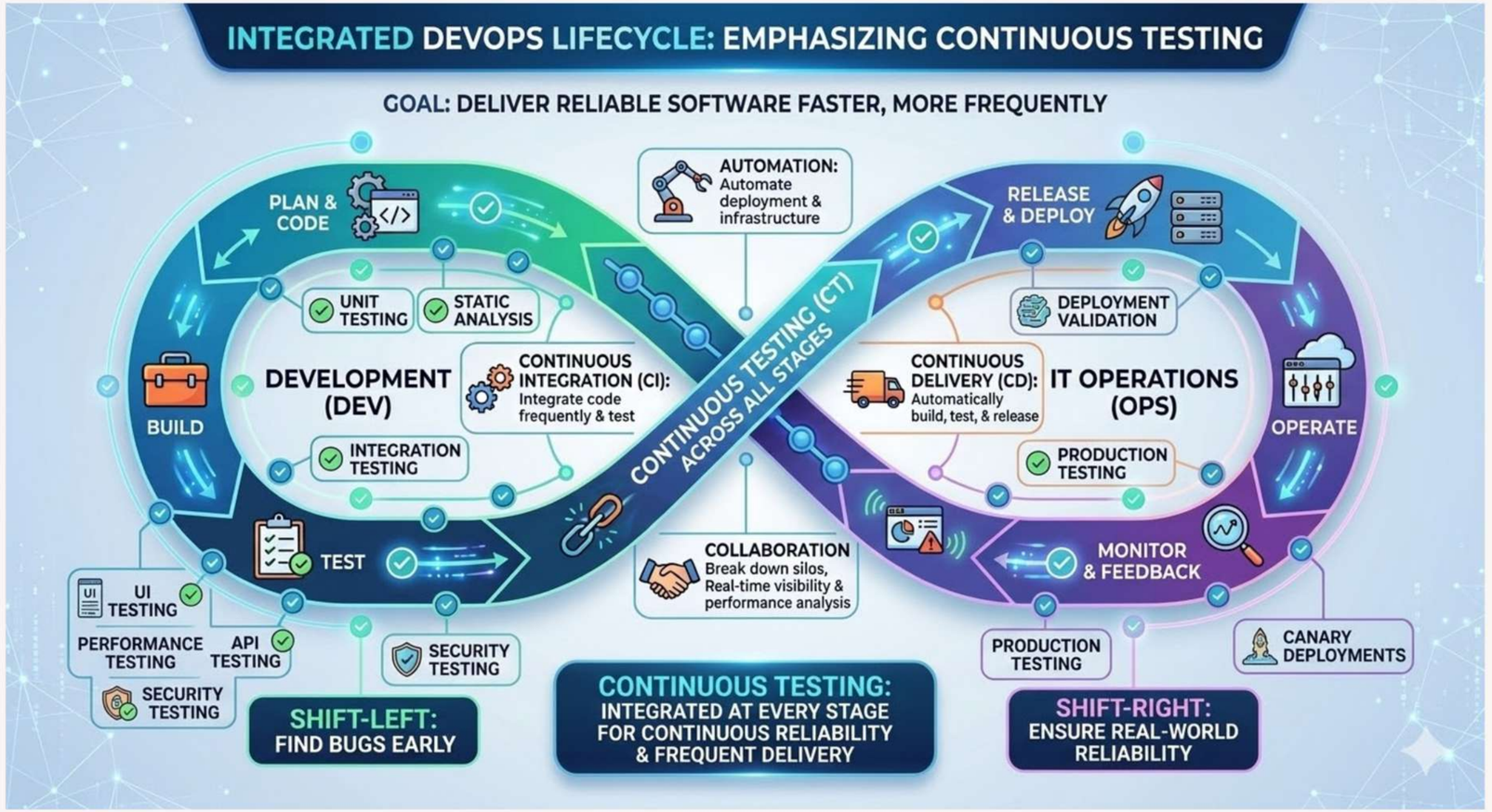
The DevOps lifecycle is continuous and iterative. Testing is not a single stage anymore. Instead, testing occurs throughout the lifecycle.

Example:

- Developers write unit tests while coding
- Automated tests run during build
- Performance tests run before deployment
- Monitoring validates behavior after release

Continuous Testing

Continuous testing means automated testing at every stage of the DevOps pipeline.
If any test fails, the pipeline stops automatically.



Testing Strategies

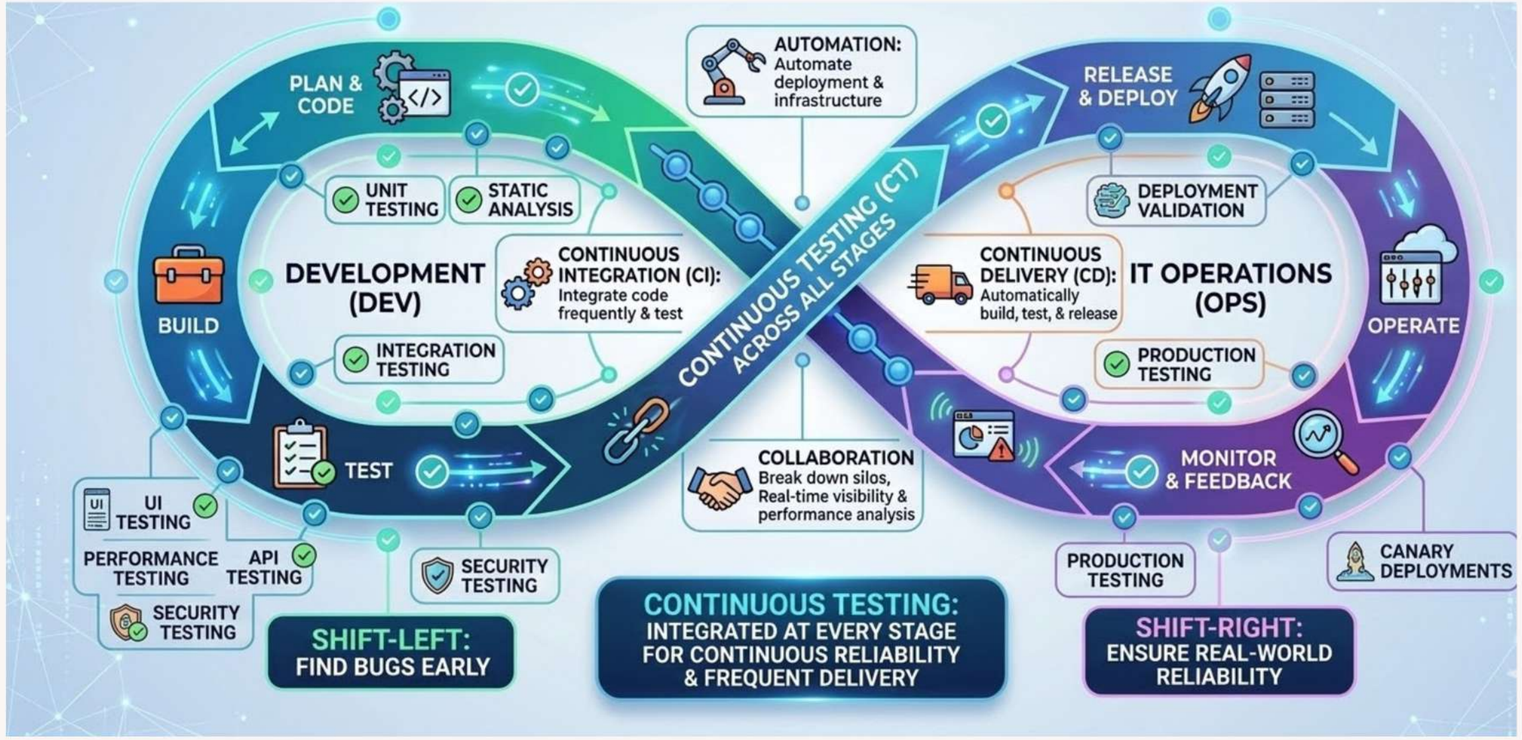
There are two major testing strategies used in Devops.

Shift-Left Testing

- Shift-Left testing means moving testing activities earlier in the development lifecycle.
- Testing begins during Requirements, Design, & Coding phases.
- The goal is to detect defects as early as possible.
- This reduces the cost and complexity of fixing bugs.

Key practices include:

- Early requirement validation
- Static code analysis
- Unit testing
- Integration testing
- Security testing during development

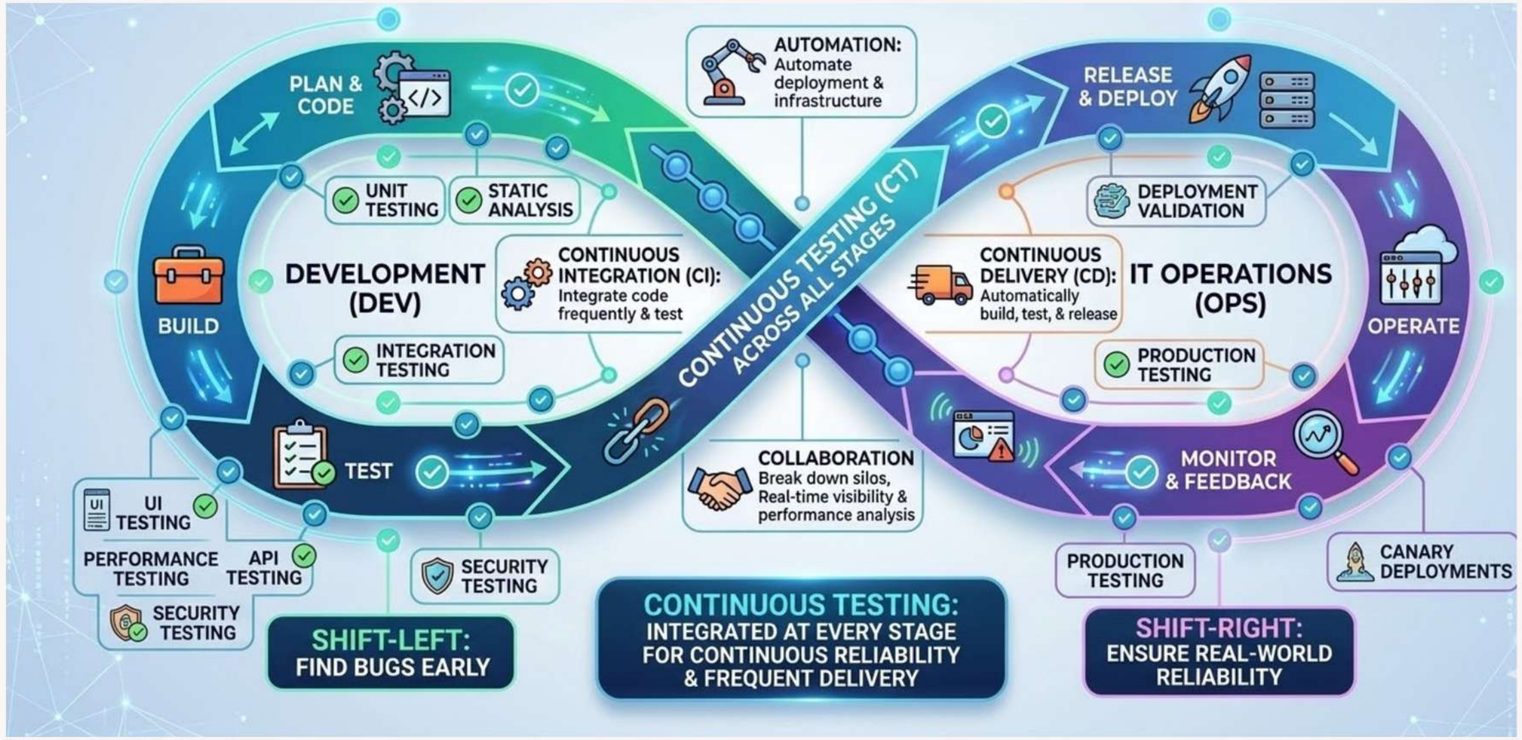


Testing Strategies

There are two major testing strategies used in Devops.

Shift-Right Testing

- Shift-Right testing focuses on testing after deployment, especially in production environments.
- The purpose is to understand how the system behaves under real-world usage conditions.
- Shift-Right testing analyzes- real user behavior, system performance, reliability under real traffic etc.
- Common Shift-Right techniques include:
  - Canary releases
  - A/B testing
  - Production monitoring
  - Chaos engineering
- Monitoring tools include:
  - Prometheus
  - Grafana
  - New Relic



Example:
A new feature is deployed to **5% of users** first before full rollout.

Shift-Left vs Shift-Right Testing

Aspect	Shift-Left Testing	Shift-Right Testing
Testing Stage	Performed during early development phases such as requirements, design, and coding.	Performed after deployment in staging or production environments.
Primary Objective	Detect defects as early as possible to reduce development cost.	Validate system behavior under real-world conditions and workloads.
Testing Environment	Development environment, CI pipelines, and staging environments.	Production or production-like environments with real user traffic.
Testing Techniques	Static analysis, unit testing, integration testing, security scanning.	Monitoring, chaos engineering, A/B testing, canary releases.
Focus Area	Code quality and early defect prevention.	System reliability, scalability, and user experience.
Tools Used	SonarQube, JUnit, Selenium	Prometheus, Grafana, Chaos Monkey
Type of Feedback	Immediate feedback to developers during coding.	Feedback from real users and system monitoring data.
Risk Reduction	Prevents defects from reaching production.	Detects unexpected failures and performance issues in production.

END OF CHAPTER

Any Questions?